LAPORAN TUGAS BESAR 1
IF3270 Pembelajaran Mesin

Feedforward Neural Network from Scratch

Disusun Oleh Kelompok LimaDua:

Amira Izani 13523143

Benedictus Nelson 13523150

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG

2026

DAFTAR ISI

DAFTAR ISI **2**
BAB I
PENDAHULUAN **3**
1.1 Deskripsi Persoalan 3
BAB II
PEMBAHASAN DAN IMPLEMENTASI **4**
2.1 Implementasi Kelas 4
2.1.1. Kelas Activation 4
2.1.2. Kelas Loss 6
2.1.3. Kelas Layer 7
2.1.4. Kelas FFNN (Feed Forward Neural Network) 9
2.2 Forward Propagation 11
2.3 Backward Propagation dan Weight Update 12
2.4 Proses Training Model 14
BAB III
PERCOBAAN DAN ANALISIS **16**
3.1 Pengaruh Width dan Depth 16
3.1.1. Width 16
3.1.2. Depth 19
3.2 Pengaruh Fungsi Aktivasi 22
3.3 Pengaruh Learning Rate 28
3.4 Pengaruh Inisialisasi Bobot 32
3.5 Pengaruh Regularisasi 42
3.6 Perbandingan Model Terbaik dengan Sklearn MLPClassifier 44
BAB IV
KESIMPULAN DAN SARAN **47**
4.1 Kesimpulan 47
4.2 Saran 48
Pembagian Tugas **49**
Lampiran **50**
Referensi **52**

BAB I
PENDAHULUAN

1.1 Deskripsi Persoalan

Pada Tugas Besar 1 mata kuliah IF3270 Pembelajaran Mesin ini, ditugaskan untuk mengimplementasi *Feedforward Neural Network* (FFNN) from scratch menggunakan bahasa pemrograman Python. Model yang dibuat dapat menerima jumlah layer serta jumlah neuron pada setiap layer, dan dapat menerima beberapa fungsi aktivasi seperti Linear, ReLU, Sigmoid, Tanh, dan Softmax.

Model yang diimplementasikan juga terdiri dari komponen-komponen yang diperlukan untuk proses pelatihan *neural network*. Model mendukung beberapa fungsi loss seperti Mean Squared Error (MSE), Binary Cross-Entropy, dan Categorical Cross-Entropy. Selain itu, perlu menyediakan mekanisme inisialisasi bobot tiap neuron dengan beberapa metode, yaitu zero initialization, random uniform, dan random normal. Setiap bobot dan bias yang dihasilkan harus disimpan oleh model beserta nilai gradiennya.

Proses *training network* dilakukan melalui implementasi forward propagation untuk menghasilkan prediksi dari data input serta backward propagation untuk menghitung gradien bobot terhadap nilai loss menggunakan konsep chain rule. Gradien tersebut kemudian digunakan untuk memperbarui bobot menggunakan metode gradient descent. Model juga dirancang untuk menangani data dalam bentuk batch serta menerima beberapa parameter pelatihan seperti batch size, learning rate, jumlah epoch, dan tingkat verbose. Selain itu, implementasi dilengkapi dengan metode regularisasi L1 dan L2.

Pengujian dilakukan menggunakan dataset *Global Student Placement & Salary Dataset* untuk memprediksi placement_status berdasarkan fitur-fitur seperti CGPA, jumlah backlog, jumlah internship, skor aptitude, dan skor komunikasi. Pengujian dilakukan dengan beberapa eksperimen untuk melihat pengaruh variasi hyperparameter seperti depth, width, fungsi aktivasi, learning rate, dan penggunaan regularisasi terhadap performa model. Hasil prediksi dan proses pelatihan kemudian dianalisis melalui perbandingan grafik training loss dan validation loss, serta dibandingkan dengan model MLPClassifier dari library scikit-learn sebagai referensi implementasi *neural network* yang sudah tersedia.

BAB II
PEMBAHASAN DAN IMPLEMENTASI

2.1 Implementasi Kelas

Modul utama disimpan dalam file ffnn.py yang terdiri dari empat kelas, yaitu Activation, Loss, Layer, dan FFNN.

2.1.1. Kelas Activation

Kelas Activation berisi kumpulan fungsi Aktivasi serta turunannya. Method bersifat static sehingga dapat dipanggil tanpa membuat Instance.

Method	Deskripsi
`linear(x)`	Fungsi aktivasi linear, di mana mengembalikan nilai input. $$Linear(x) = x$$
`linear_derivative(x)`	Turunan linear. Mengembalikan matriks bernilai 1. $$\frac{d\big(Linear(x)\big)}{dx} = 1$$
`relu(x)`	Fungsi aktivasi ReLU, di mana jika nilai input negatif mengembalikan nol, dan nilai positif mengembalikan nilai input. $$ReLU(x) = \max(0, x)$$
`relu_derivative(x)`	Turunan ReLU. mengembalikan nilai 1 jika x > 0 dan 0 jika x <= 0. $$\frac{d\big(ReLU(x)\big)}{dx} = \begin{cases} 0, & x \leq 0 \\ 1, & x > 0 \end{cases}$$
`sigmoid(x)`	Fungsi aktivasi sigmoid:

	$$\sigma(x) = \frac{1}{1+e^{-x}}$$
`sigmoid_derivative(x)`	Turunan Sigmoid: $$\frac{d(\sigma(x))}{dx} = \sigma(x)(1-\sigma(x))$$
`tanh(x)`	Fungsi aktivasi Hyperbolic Tangent (tanh): $$\sigma(x) = \frac{1}{1+e^{-x}}$$
`tanh_derivative(x)`	Turunan Hyperbolic Tangent (tanh): $$\tanh'(x) = 1 - \tanh^2(x)$$
`softmax(x)`	Fungsi aktivasi softmax: $$\text{Untuk vector } \vec{x} = (x_1, x_2, \dots, x_n) \in \mathbb{R}^n,$$ $$softmax(\vec{x})_i = \frac{e^{x_i}}{\sum_{j=1}^{n} e^{x_j}}$$ Dilakukan pengurang max(x) per sampel (axis = 1) untuk mencegah overflow.
`softmax_derivative(x)`	Turunan softmax: $$D_j S_i = S_i(\delta_{ij} - S_j)$$ Di mana, $$\delta_{ij} = \begin{cases} 1 & i = j \\ 0 & i \neq j \end{cases}$$ Turunan akan membentuk Jacobian matriks `Jacobian = diag(s) - ss`^{T}
`leaky_relu(x, alpha = 0.01)`	Fungsi Aktivasi leaky ReLU:

	$$f(x) = \begin{cases} x, & x > 0 \\ \alpha x, & x \leq 0 \end{cases}$$ Mengatasi "Dying ReLU" dimana output terjebak bernilai nol dengan memberikan gradien kecil (α).
`leaky_relu_derivative(x, alpha = 0.01)`	Turunan Leaky ReLU: 1 jika x > 0, α jika x <= 0.
`elu(x, alpha = 0.01)`	Fungsi Aktivasi ELU (Exponential Linear Unit) $$f(x) = \begin{cases} x, & x > 0 \\ \alpha(e^x - 1), & x \leq 0 \end{cases}$$
`elu_derivative(x, alpha = 0.01)`	Turunan ELU: 1 jika x > 0, dan α (e^x) jika x <= 0.

2.1.2. Kelas Loss

Kelas Loss berfungsi sebagai kumpulan fungsi loss beserta turunannya. Method bersifat static sehingga dapat dipanggil tanpa membuat Instance. Turunan yang diimplementasikan pada kelas ini dihitung terhadap nilai prediksi model bukan langsung terhadap bobot karena gradien terhadap bobot dihitung nanti melalui proses backpropagation menggunakan chain rule pada setiap layer.

Method	Deskripsi
`mse(y_true, y_pred)`	Fungsi Mean Squared Error digunakan untuk regresi $$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$
`mse_derivative(y_true, y_pred)`	Turunan MSE: di mana n merupakan jumlah sampel dalam batch $$\frac{\partial \mathcal{L}_{MSE}}{\partial W} = -\frac{2}{n}\sum_{i=1}^{n}\frac{\partial \mathcal{L}_{MSE}}{\partial \hat{y}_i}\frac{\partial \hat{y}_i}{\partial W} = -\frac{2}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)\frac{\partial \hat{y}_i}{\partial W}$$
`bce(y_true, y_pred)`	Fungsi Binary Cross-Entropy, di man y_pred dibatasi interval [$1e^{-15}$, $1-1^{-15}$] menghindari loss menjadi tak

	terhingga. $$\mathcal{L}_{BCE} = -\frac{1}{n}\sum_{i=1}^{n}(y_i \log \hat{y}_i + (1 - y_i)\log(1 - \hat{y}_i))$$ y_i = Actual binary label (0 or 1) $\hat{y}_i$ = Predicted value of y_i n = Batch size
`bce_derivative(y_true, y_pred)`	Turunan BCE $$\frac{\partial \mathcal{L}_{BCE}}{\partial W} = -\frac{1}{n}\sum_{i=1}^{n}\frac{\partial \mathcal{L}_{BCE}}{\partial \hat{y}_i}\frac{\partial \hat{y}_i}{\partial W} = -\frac{1}{n}\sum_{i=1}^{n}\frac{\hat{y}_i - y_i}{\hat{y}_i(1 - \hat{y}_i)}\frac{\partial \hat{y}_i}{\partial W}$$
`cce(y_true, y_pred)`	Fungsi Categorical Cross-Entropy berfungsi untuk klasifikasi multi-kelas dengan y_true berformat one-hot encoding. $$\mathcal{L}_{CCE} = -\frac{1}{n}\sum_{i=1}^{n}\sum_{j=1}^{C}(y_{ij} \log \hat{y}_{ij})$$ y_{ij} = Actual value of instance i for class j $\hat{y}_{ij}$ = Predicted value of y_{ij} C = Number of classes n = Batch size
`cce_derivative(y_true, y_pred)`	Turunan CCE $$\frac{\partial \mathcal{L}_{CCE}}{\partial W} = -\frac{1}{n}\sum_{i=1}^{n}\sum_{j=1}^{C}\frac{\partial \mathcal{L}_{CCE}}{\partial \hat{y}_{ij}}\frac{\partial \hat{y}_{ij}}{\partial W} = -\frac{1}{n}\sum_{i=1}^{n}\sum_{j=1}^{C}\frac{y_{ij}}{\hat{y}_{ij}}\frac{\partial \hat{y}_{ij}}{\partial W}$$

2.1.3. Kelas Layer

Kelas layer berfungsi sebagai representasi satu fully connected (dense) layer. Setiap layer menyimpan matriks bobot, vektor bias, fungsi aktivasi, serta nilai cache yang diperlukan untuk forward dan backward propagation.

Method/Atribut	Deskripsi
`input_size`	Tipe integer, jumlah neuron input pada layer saat ini.

`output_size`	Tipe integer, jumlah neuron output pada layer saat ini.
`activation_name`	Tipe string, nama fungsi aktivasi yang digunakan (contohnya, ‘relu’, ‘softmax’)
`weights_init_name`	Tipe string, nama inisialisasi bobot yang digunakan.
`init_params`	Tipe dict, Parameter tambahan inisialisasi yang disimpan, yaitu seed, mean, variance, lower_bound, upper_bound
`weights`	Shape ndarray (n_in, n_out), Matriks bobot layer di mana diinisialisasi sesuai dengan `weight_init`. Atribut ini diperbarui tiap backward propagation
`biases`	Shape ndarray (1, n_out). Vektor bias layer di mana diperbarui tiap backward propagation
`d_weights`	Shape ndarray (n_in, n_out). Nilai gradien bobot dihitung dan disimpan saat backward propagation
`d_biases`	Shape ndarray (1, n_out). Nilai gradien bias di mana dihitung dan disimpan saat backward propagation.
`input`	Shape ndarray (batch, n_in). Nilai input yang masuk ke layer saat proses forward propagation. Nilai disimpan agar dapat digunakan untuk menghitung d_weights pada proses backward propagation.
`z`	Shape ndarray (batch, n_out). Nilai sebelum fungsi aktivasi $z = \sum(w.x) + b$. Nilai disimpan untuk menghitung turunan fungsi aktivasi pada backward propagation.
`output`	Shape ndarray (batch, n_out). Nilai hasil forward propagation $z = f(z)$ yang diteruskan untuk input layer setelahnya.
`__init__(self, input_size, output_size, activation='relu', weight_init='random_normal', **kwargs)`	Fungsi kontruktor untuk menetapkan jumlah input dan output neuron, fungsi aktivasi dan inisialisasi bobot yang digunakan, serta nilai sementara yang digunakan untuk backward propagation dan layer setelahnya.
`forward(self, input_data)`	Melakukan proses forward propagation. Fungsi menyimpan input, menghitung nilai z, lalu menghasilkan output dengan menerapkan fungsi aktivasi dan mengembalikan output dari layer
`backward(self, output_gradient,`	Proses backward propagation. Prosesnya menghitung gradien terhadap nilai sebelum aktivasi z, menghitung

`learning_rate, l1_lambda=0, l2_lambda=0, is_output_layer=False, loss_name=None)`	d_weights dan d_biases, menambahkan gradien regularisasi L1/l2, serta memperbarui nilai bobot dan bias. Akhirnya akan menghasilkan input_gradien yang diteruskan ke layer sebelumnya

terdapat metode inisialisasi bobot yang tersimpan pada fungsi __init__ di kelas layer. Berikut inisialisasi bobot yang digunakan:

Nama Inisialisasi Bobot	Deskripsi
`'zero'`	Bobot dan bias diinisialisasi dengan nilai nol
`'random_uniform'`	Bobot diinisialisasi secara acak dari distribusi uniform dalam rentang tertentu.
`'random_normal'`	Inisialisasi bobot dari distribusi normal dengan mean dan variance tertentu
`'xavier'`	Inisialisasi bobot dengan metode xavier $x = \sqrt{\frac{6}{n_{inputs} + n_{outputs}}}$ • x is calculated using above formula. • n_{inputs} : Number of Input in the input layer • n_{output} : Number of Output in the Output layer For each weight in network we draw a random value w from a uniform distribution in the range [-x, x].
`'he'`	Inisialisasi bobot dengan metode He/Kaiming $W \rightarrow \mathcal{N}\left(0, \sqrt{\frac{2}{n}}\right)$

2.1.4. Kelas FFNN (Feed Forward Neural Network)

Pada kelas FFNN, dilakukan integrasi semua layer menjadi sebuah *neural network.* Kelas ini merupakan suatu interface untuk membangun arsitektur, melatih model, membuat prediksi, serta menyimpan dan memuat model.

Method/Atribut	Deskripsi
`layers`	Daftar semua objek layer yang membentuk suatu *network* yang sudah terurut dari input layer hingga output layer.

`loss_name`	Nama fungsi loss yang digunakan.
`loss_func`	Untuk referensi ke fungsi loss yang sedang aktif (merujuk ke kelas Loss) yang nanti ditetapkan saat proses compile()
`loss_derivative`	Referensi ke fungsi turunan loss yang sedang aktik (merujuk ke kelas Loss) untuk menghitung gradien awal backward propagation.
`l1_lambda`	Regularisasi L1. Jika tidak digunakan bernilai nol.
`l2_lambda`	Regularisasi L2. Jika tidak digunakan bernilai nol.
`__init__(self)`	Konstruktor. Inisialisasi list layers dan atribut
`add(self, layer):`	Menambahkan objek layer ke dalam *network* dengan append secara berurutan.
`compile(self, loss='mse', l1_lambda=0, l2_lambda=0)`	Konfigurasi model sebelum training: menetapkan fungsi loss, turunan, dan apakah menggunakan parameter regularisasi.
`forward(self, X)`	Proses forward propagation. Meneruskan input X ke seluruh layer secara berurutan, di mana output dari tiap layer menjadi input untuk layer selanjutnya.
`backward(self, X, y_true, y_pred, learning_rate)`	Proses backward propagation dengan menghitung output_gradien dari fungsi loss, lalu menyebarkannya ke tiap layer mulai dari layer output menuju layer input. Pada tiap layer, gradien dihitung dan bobot diperbarui.
`fit(self, X_train, y_train, epochs, batch_size, learning_rate, validation_data=None, verbose=1)`	Model training, di mana tiap epoch dilakukan shuffling data yang diproses tiap mini-batch dari proses forward, loss, lalu backward dan update weight, serta menghitung val-loss. Ketika Verbose 0 maka akan diam dan ketika Verbose 1 akan menampilkan progress loss. Mengembalikan dictionary {'loss': [...], 'val_loss': [...]}.
`predict(self, X)`	Prediksi pada data X dengan memanggil fungsi forward(X)
`plot_weight_distribution(self, layer_indices=None)`	Menampilkan histogram distribusi bobot.
`plot_gradient_distribution(self, layer_indices=None)`	Menampilkan histogram distribusi gradien bobot. Dipanggil saat minimal melakukan forward dan backward propagation.

`save(self, filepath)`	Menyimpan model ke file .pkl yang berisi layers, loss_name, l1_lambda, dan l2_lambda
`load(self, filepath)`	Memuat model dari file .pkl

2.2 Forward Propagation

Forward propagation merupakan proses menghitung output *network* dari data input. Data input diteruskan satu arah dari *input layer* melalui *hidden layer* hingga ke *output layer.* Tiap layer melakukan operasi menghitung nilai sebelum fungsi aktivasi (nilai z) lalu menerapkan fungsi aktivasi pada z. Operasi dilakukan dalam bentuk perkalian matriks (batch processing) sehingga banyak sampel data diproses sekaligus tanpa loop per instance.

```
def forward(self, input_data):
    self.input = input_data
    self.z = np.dot(self.input, self.weights) + self.biases
    self.output = self.activation(self.z)
    return self.output
```

Fungsi ini berada di kelas layer. Langkah-langkah yang dilakukan:

1. Input disimpan ke self.Input untuk menghitung d_weights pada proses backward nanti.
2. Menghitung nilai sebelum aktivasi z menggunakan np.dot untuk operasi matriks batch
3. Nilai z digunakan dalam fungsi aktivasi lalu dikembalikan.

```
def forward(self, X):
    output = X
    for layer in self.layers:
        output = layer.forward(output)
    return output
```

Fungsi ini berada di kelas FFNN, di mana kelas ini menghubungkan semua layer secara berurutan. Dalam fungsi ini, output tiap layer langsung diteruskan ke layer berikutnya sebagai input.

2.3 Backward Propagation dan Weight Update

Pada proses backward propagation dilakukan penghitungan gradien fungsi loss terhadap tiap bobot dalam network dengan menggunakan prinsip chain rule. Gradien kemudian akan digunakan untuk memperbarui bobot secara bertahap agar nilai loss semakin kecil. Perhitungan gradien dilakukan dengan memeriksa hubungan dari output kembali ke bobot yang ingin diperbarui. Gradien terhadap suatu bobot diperoleh dari hasil perkalian gradien loss terhadap output, turunan fungsi aktivasi terhadap nilai sebelum aktivasi, serta nilai input yang masuk ke dalam bobot.

```python
def backward(self, output_gradient, learning_rate, l1_lambda=0, l2_lambda=0, is_output_layer=False, loss_name=None):
    if is_output_layer and self.activation_name == 'softmax' and loss_name == 'categorical_crossentropy':
        d_z = output_gradient
    elif is_output_layer and self.activation_name == 'sigmoid' and loss_name == 'binary_crossentropy':
        d_z = output_gradient
    elif self.activation_name == 'softmax':
        # Jacobian-vector product without materializing per-sample Jacobians.
        s = self.output
        dot = np.sum(output_gradient * s, axis=1, keepdims=True)
        d_z = s * (output_gradient - dot)
    else:
        d_z = output_gradient * self.activation_derivative(self.z)

    input_gradient = np.dot(d_z, self.weights.T)
    self.d_weights = np.dot(self.input.T, d_z)
    self.d_biases = np.sum(d_z, axis=0, keepdims=True)

    if l1_lambda > 0:
        self.d_weights += l1_lambda * np.sign(self.weights)
    if l2_lambda > 0:
        self.d_weights += l2_lambda * self.weights

    # Weight update
    self.weights -= learning_rate * self.d_weights
    self.biases -= learning_rate * self.d_biases

    return input_gradient
```

Berikut merupakan implementasi backward propagation pada kelas layer:

1. Pertama-tama menghitung gradien terhadap Z. Fungsi aktivasi seperti ReLU, sigmoid, Tanh, dan linear, nilai diperoleh dari perkalian antara gradien layer berikutnya dengan turunan fungsi aktivasi. Pada fungsi Aktivasi softmax, pembentukan jacobian matriks dibentuk melalui operasi vektor langsung.
2. Gradien bobot dihitung menggunakan input yang masuk ke layer saat itu sedangkan gradien bias dengan menjumlahkan nilai gradien pada setiap neuron. Proses dilakukan pada seluruh data dalam satu *mini-batch.*
3. Regularisasi L1 dan L2 dapat ditambahkan pada gradien bobot. L1 akan membuat bobot menjadi nol sedangkan L2 menjaga agar bobot kecil dan stabil.
4. Gradien dihitung yang akan diteruskan ke layer sebelumnya dengan meneruskan melalui bobot saat ini. Nilai akan menjadi input gradien layer sebelumnya selama proses backward.
5. Terakhir, nilai bobot dan bias diperbarui menggunakan learning rate.

```python
def backward(self, X, y_true, y_pred, learning_rate):
    if self.loss_name == 'categorical_crossentropy' and self.layers[-1].activation_name == 'softmax':
        # Combined derivative simplification
        output_gradient = (y_pred - y_true) / y_true.shape[0]
    elif self.loss_name == 'binary_crossentropy' and self.layers[-1].activation_name == 'sigmoid':
        output_gradient = (y_pred - y_true) / y_true.shape[0]
    else:
        output_gradient = self.loss_derivative(y_true, y_pred)

    for i in reversed(range(len(self.layers))):
        layer = self.layers[i]
        is_output_layer = (i == len(self.layers) - 1)
        output_gradient = layer.backward(
            output_gradient,
            learning_rate,
            self.l1_lambda,
            self.l2_lambda,
            is_output_layer,
            self.loss_name
        )
```

Pada kelas FFNN proses backward propagation dimulai dengan menentukan nilai gradien awal dari output layer:

1. Dilakukan pengecekan kombinasi fungsi loss dan fungsi aktivasi yang digunakan.
2. Jika output layer menggunakan softmax dengan CCE atau sigmoid dengan BCE, maka gradien dapat langsung dihitung dari selisih antara prediksi dan label.
3. Selain dari kombinasi tersebut, gradien dihitung dengan fungsi turunan loss yang umum
4. Setelah gradien awal diperoleh, nilai tersebut akan diteruskan ke tiap layer dari layer output sampai menuju layer utama. Tiap layer akan menghitung gradiennya masing-masing dan mengembalikan gradien untuk layer sebelumnya.

2.4 Proses Training Model

Metode fit() menggabungkan seluruh proses training menjadi satu loop training yang berjalan tiap epoch.

```python
def fit(self, X_train, y_train, epochs, batch_size, learning_rate, validation_data=None, verbose=1):
    history = {'loss': [], 'val_loss': []}
    import sys

    num_samples = X_train.shape[0]

    for epoch in range(epochs):
        # Shuffle data for stochasticity
        indices = np.arange(num_samples)
        np.random.shuffle(indices)
        X_shuffled = X_train[indices]
        y_shuffled = y_train[indices]

        epoch_loss = 0
        # Batch processing
        for i in range(0, num_samples, batch_size):
            X_batch = X_shuffled[i:i + batch_size]
            y_batch = y_shuffled[i:i + batch_size]

            # Forward Pass
            y_pred_batch = self.forward(X_batch)

            # Calculate Loss
            loss = self.loss_func(y_batch, y_pred_batch)

            # Add Regularization Loss
            l1_penalty = 0
            l2_penalty = 0
            for layer in self.layers:
                if self.l1_lambda > 0:
                    l1_penalty += self.l1_lambda * np.sum(np.abs(layer.weights))
                if self.l2_lambda > 0:
                    l2_penalty += 0.5 * self.l2_lambda * np.sum(layer.weights ** 2)

            loss += l1_penalty + l2_penalty
            epoch_loss += loss * len(X_batch)
```

```
        # Backward Pass
        self.backward(X_batch, y_batch, y_pred_batch, learning_rate)

        if verbose == 1:
            progress = int(20 * min(i + batch_size, num_samples) / num_samples)
            bar = '=' * progress + '.' * (20 - progress)
            sys.stdout.write(f"\rEpoch {epoch+1}/{epochs} [{bar}]")
            sys.stdout.flush()

    epoch_loss /= num_samples
    history['loss'].append(epoch_loss)

    # Validation
    val_loss = None
    if validation_data:
        X_val, y_val = validation_data
        y_val_pred = self.forward(X_val)
        val_loss = self.loss_func(y_val, y_val_pred)
        history['val_loss'].append(val_loss)

    if verbose == 1:
        val_msg = f"- val_loss: {val_loss:.4f}" if validation_data else ""
        print(f" - loss: {epoch_loss:.4f} {val_msg}")

return history
```

Langkah-langkah implementasi model training:

1. Shuffle Data: Data training diacak tiap epoch agar model training pada data yang tidak tetap.
2. Pembagian mini-batch: Data dibagi menjadi beberapa batch lalu tiap batch diproses secara terpisah
3. Forward propagation: memanggil fungsi forward untuk menghasilkan prediksi pada tiap batch
4. Penghitungan loss: Dari hasil prediksi, loss akan dihitung, kemudian ditambahkan regularisasi L1/L2 jika digunakan.
5. Backward Propagation: Dilakukan penghitungan gradien dan memperbarui bobot pada tiap layer
6. Average loss per epoch: Setelah menjalankan proses looping, loss dari tiap batch akan dikumpulkan dan dirata-ratakan untuk mendapatkan nilai loss pada satu epoch.
7. Perhitungan validation loss: Loss akan dihitung menggunakan forward propagation tanpa update bobot jika terdapat data validasi
8. Penyimpanan nilai loss dan validation loss: nilai loss dan validation loss disimpan untuk membandingkan performa model antar percobaan.
9. Verbose: Jika aktif, akan menampilkan proses dan nilai loss tiap epoch.

BAB III
PERCOBAAN DAN ANALISIS

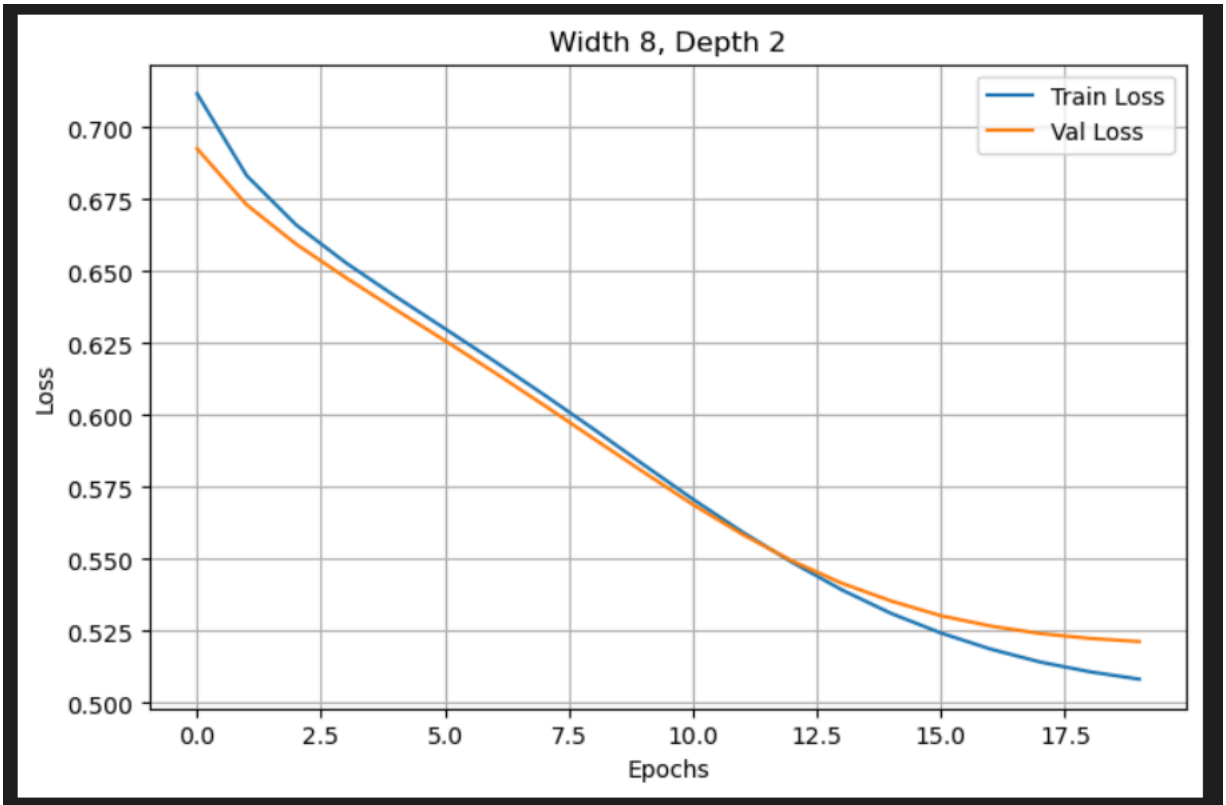
3.1 Pengaruh Width dan Depth

3.1.1. Width

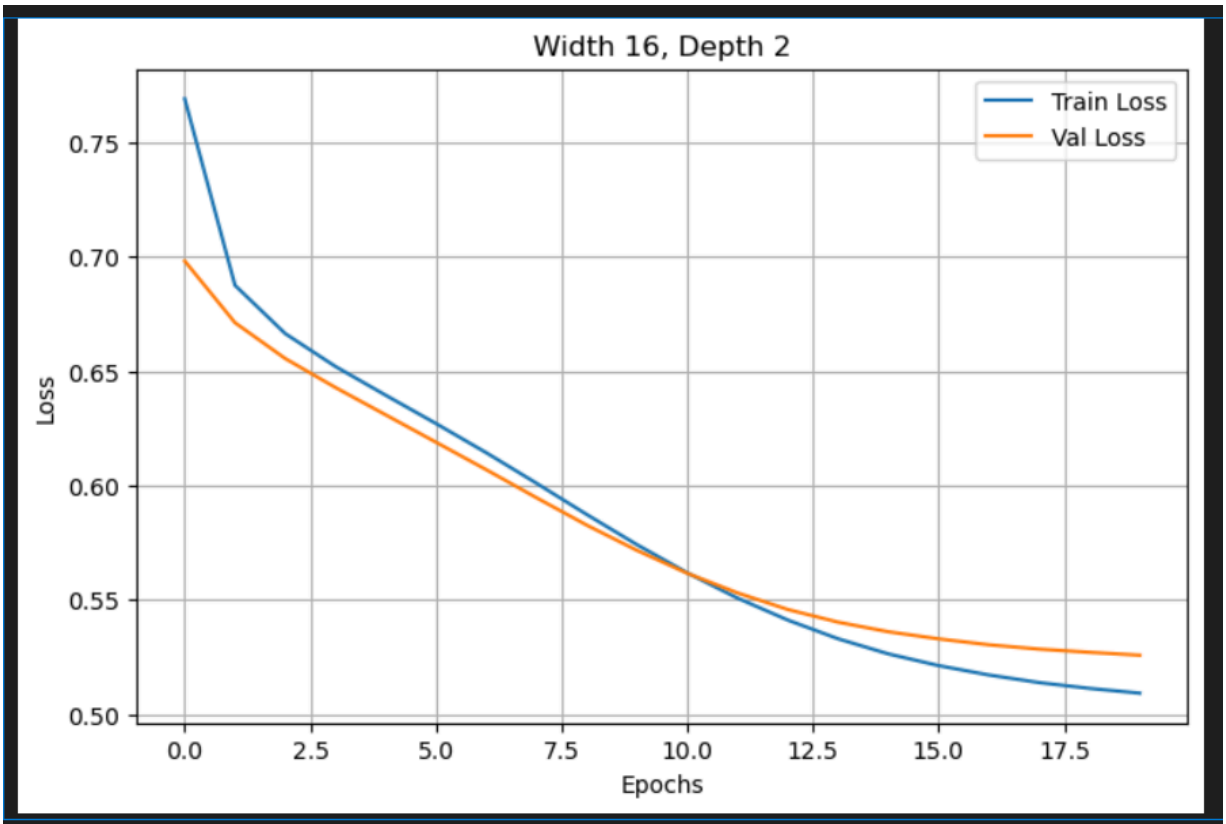
Informasi

- Depth hidden layer dibuat tetap = 2
- Width yang diuji = 8, 16, 32
- Metrik yang dibandingkan = accuracy, F1, log loss pada validation set

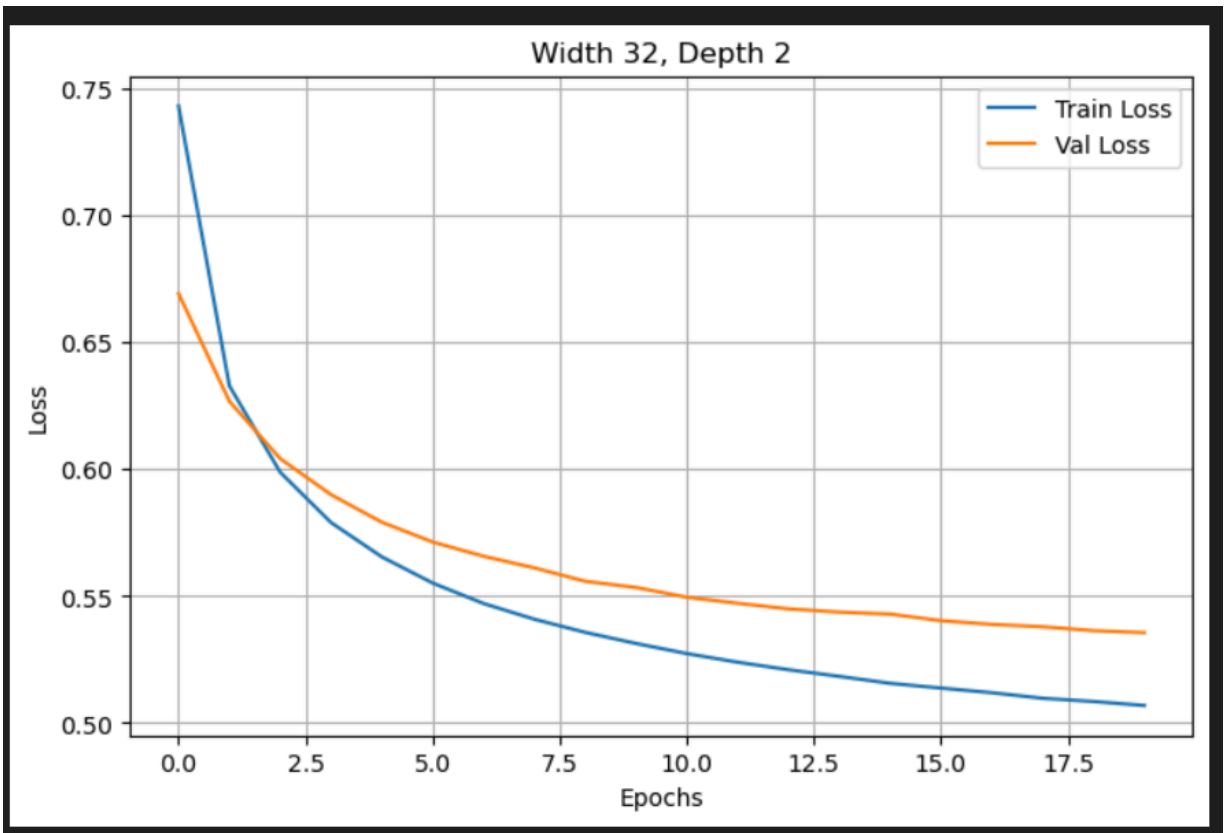
Grafik




```
--- Width 8, Depth 2 (Validation) ---
Accuracy : 0.7493
F1 Score : 0.8064
Log Loss : 0.5213
-------------------------------------
```




```
--- Width 16, Depth 2 (Validation) ---
Accuracy : 0.7507
F1 Score : 0.8072
Log Loss : 0.5257
```




```
--- Width 32, Depth 2 (Validation) ---
Accuracy : 0.7453
F1 Score : 0.8027
Log Loss : 0.5356
-------------------------------------
```

Width	Depth Tetap	Accuracy	F1-score	Log Loss
8	2	0.7493	0.8064	0.5213
16	2	0.7507	0.8072	0.5257
32	2	0.7453	0.8027	0.5356

Analisis

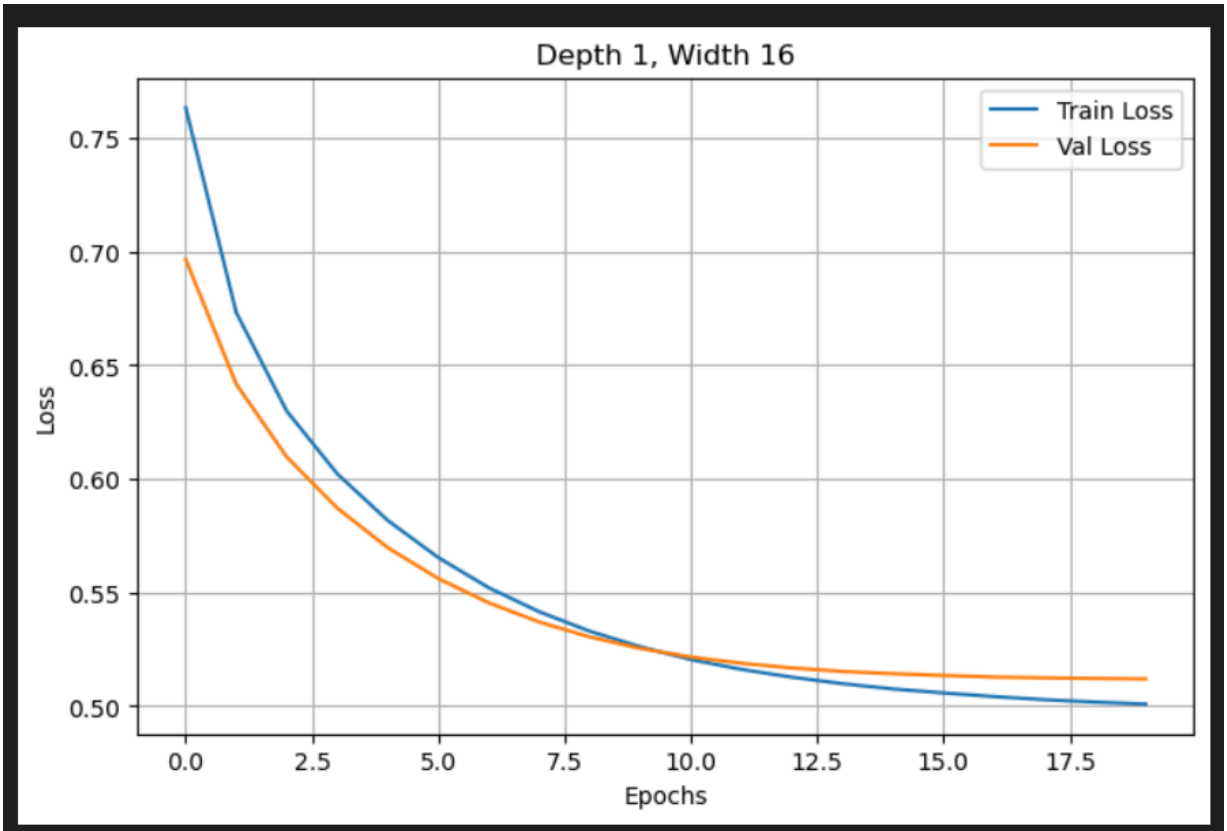
- Width 16 paling seimbang
- Width 32 tidak memberi peningkatan, malah log loss memburuk
- Width terlalu besar belum tentu lebih baik

3.1.2. Depth

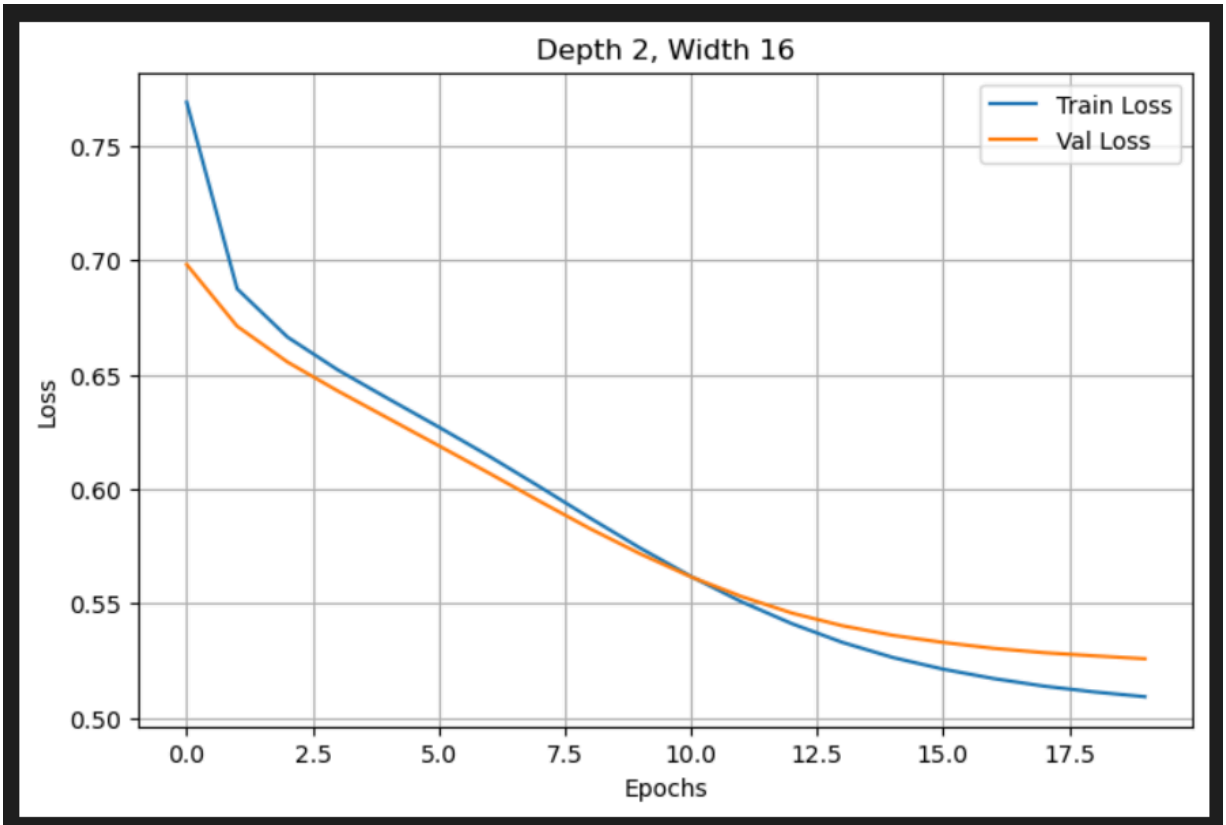
Informasi

- Width dibuat tetap = 16
- Depth yang diuji = 1, 2, 3

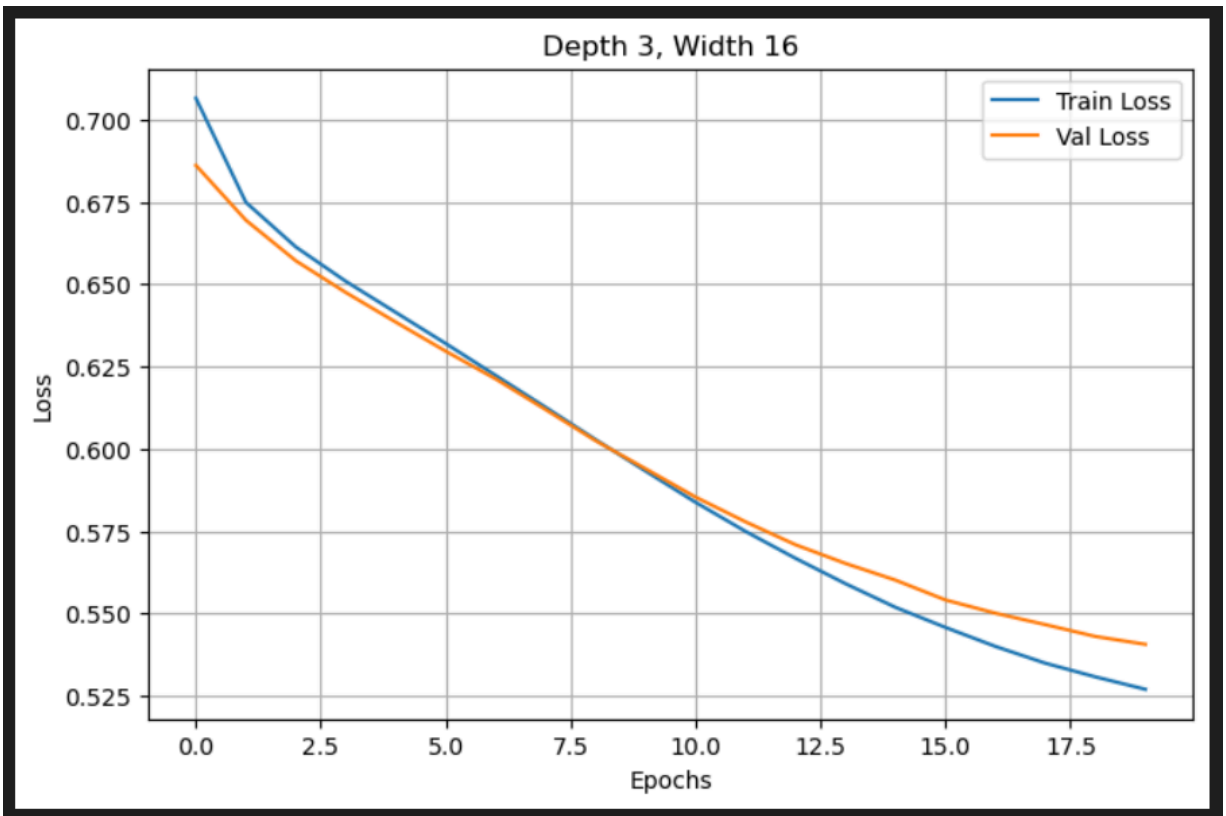
Grafik




```
--- Depth 1, Width 16 (Validation) ---
Accuracy : 0.7500
F1 Score : 0.8058
Log Loss : 0.5120
-------------------------------------
```




```
--- Depth 2, Width 16 (Validation) ---
Accuracy : 0.7507
F1 Score : 0.8072
Log Loss : 0.5257
-------------------------------------
```




```
--- Depth 3, Width 16 (Validation) ---
Accuracy : 0.7433
F1 Score : 0.8041
Log Loss : 0.5406
-------------------------------------
```

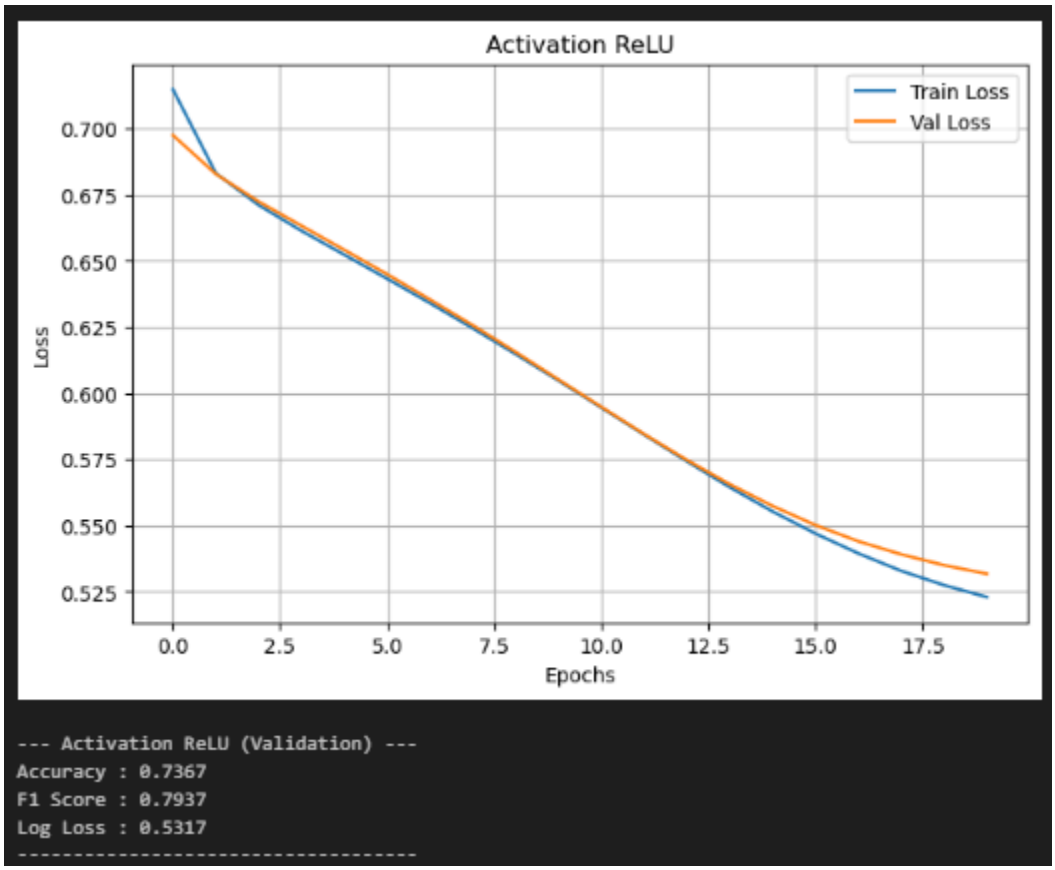
Depth	Width Tetap	Accuracy	F1-score	Log Loss
1	16	0.7500	0.8058	0.5120
2	16	0.7507	0.8072	0.5257
3	16	0.7433	0.8041	0.5406

Analisis

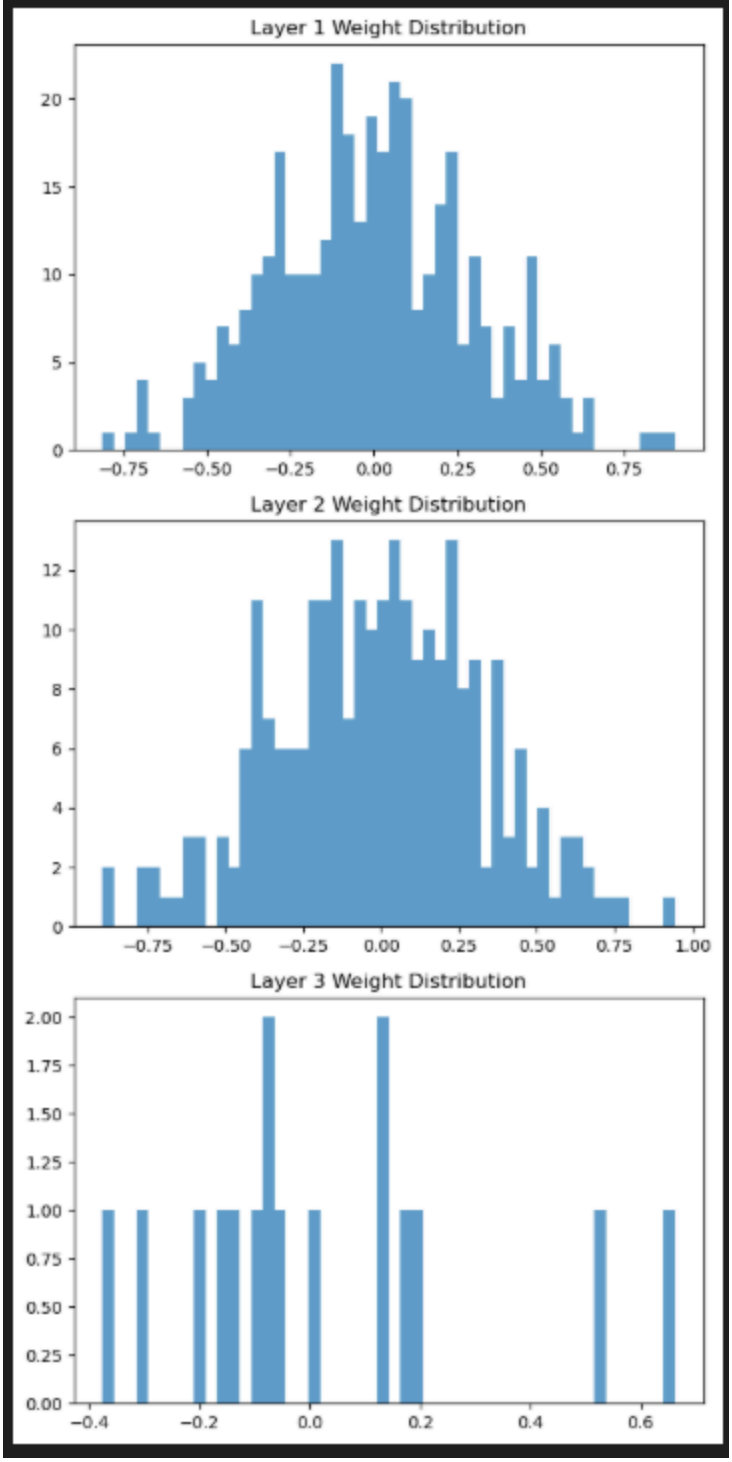
- Depth 2 sedikit lebih baik untuk accuracy/F1
- Depth 3 justru menurun
- Penambahan layer tidak otomatis meningkatkan performa

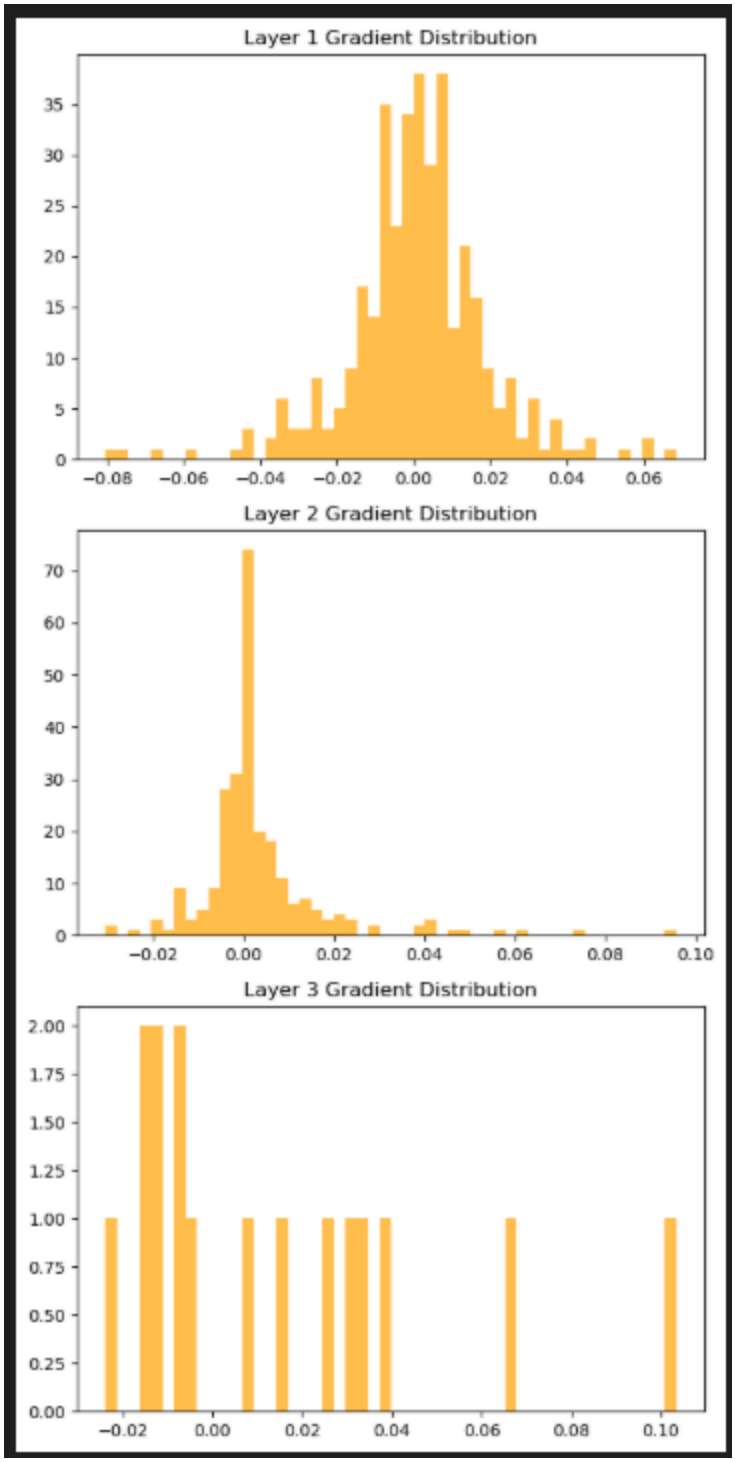
3.2 Pengaruh Fungsi Aktivasi

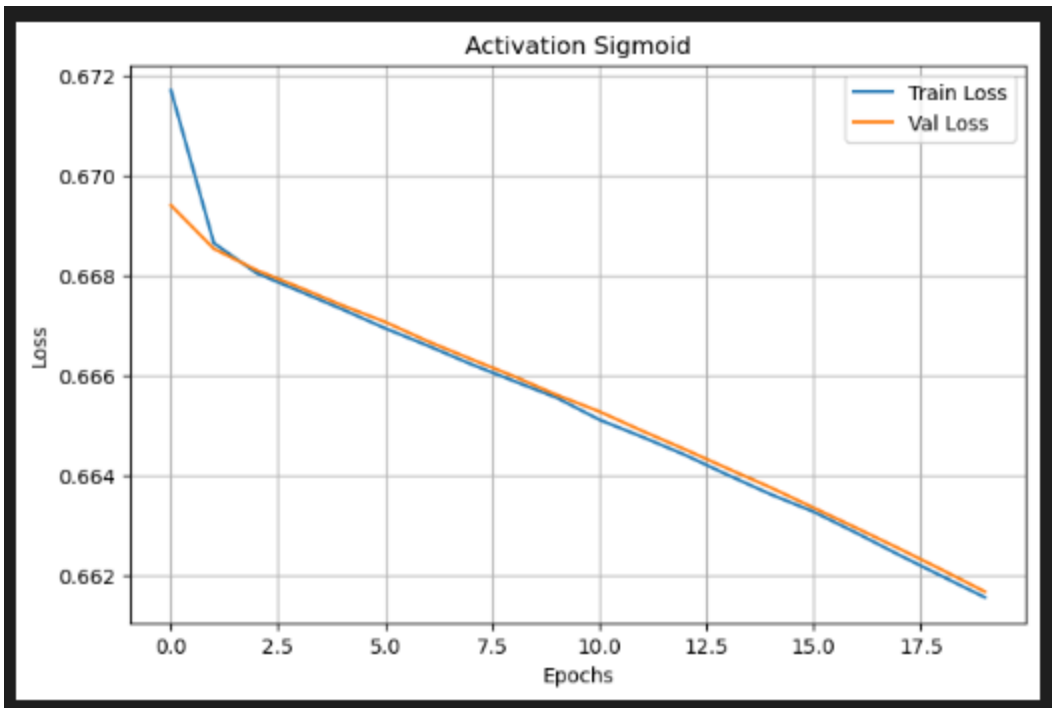
Informasi
Pada percobaan ini dilakukan pengujian pengaruh fungsi aktivasi pada hidden layer terhadap performa model FFNN. Fungsi aktivasi yang diuji meliputi ReLU, Sigmoid, Tanh, Linear, Leaky ReLU, dan ELU, sedangkan fungsi aktivasi pada output layer tetap menggunakan sigmoid karena tugas yang diselesaikan merupakan klasifikasi biner. Untuk memperjelas perbandingan, grafik yang ditampilkan secara representatif adalah Sigmoid dan Linear, disertai distribusi bobot dan gradien untuk melihat kestabilan proses pembelajaran.
Grafik



```
Plotting Weights and Gradients distribution (ReLU):
```

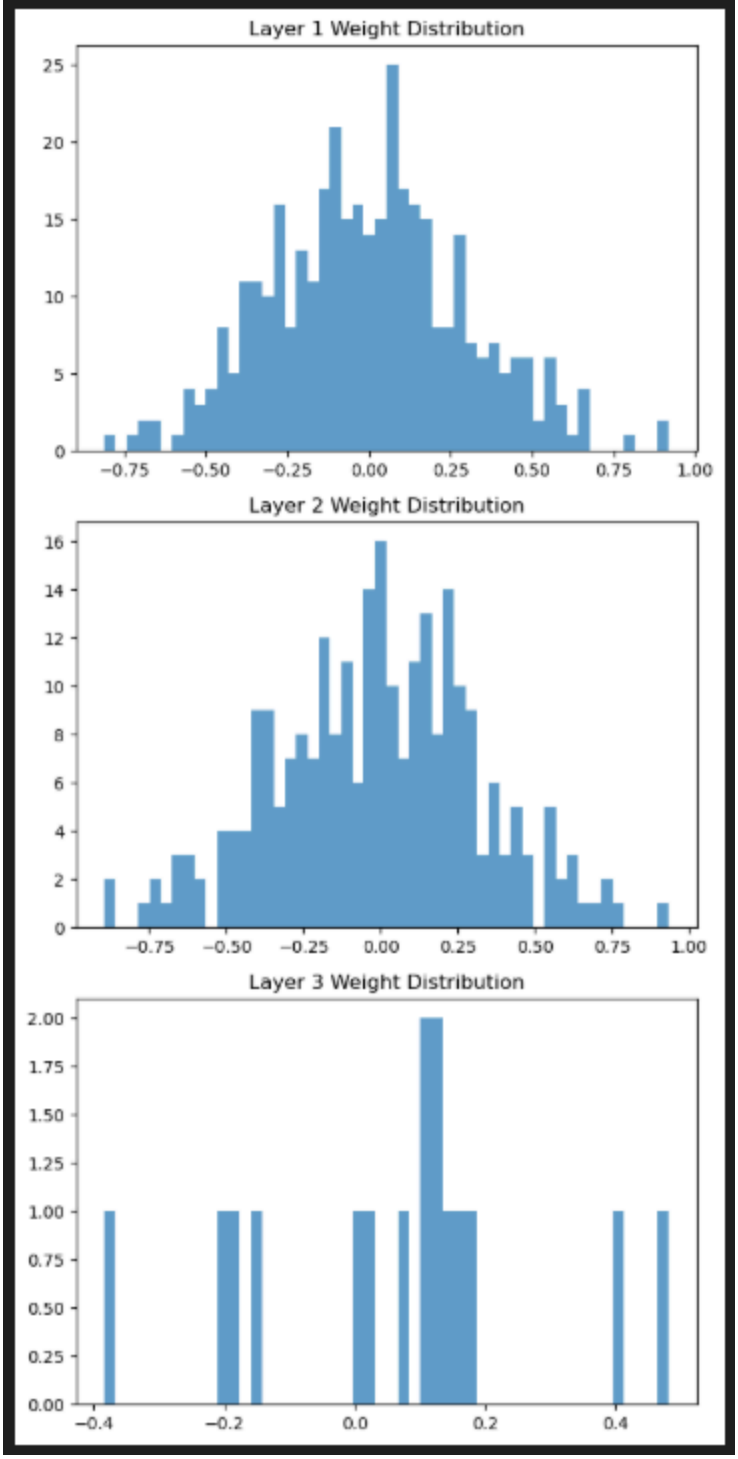


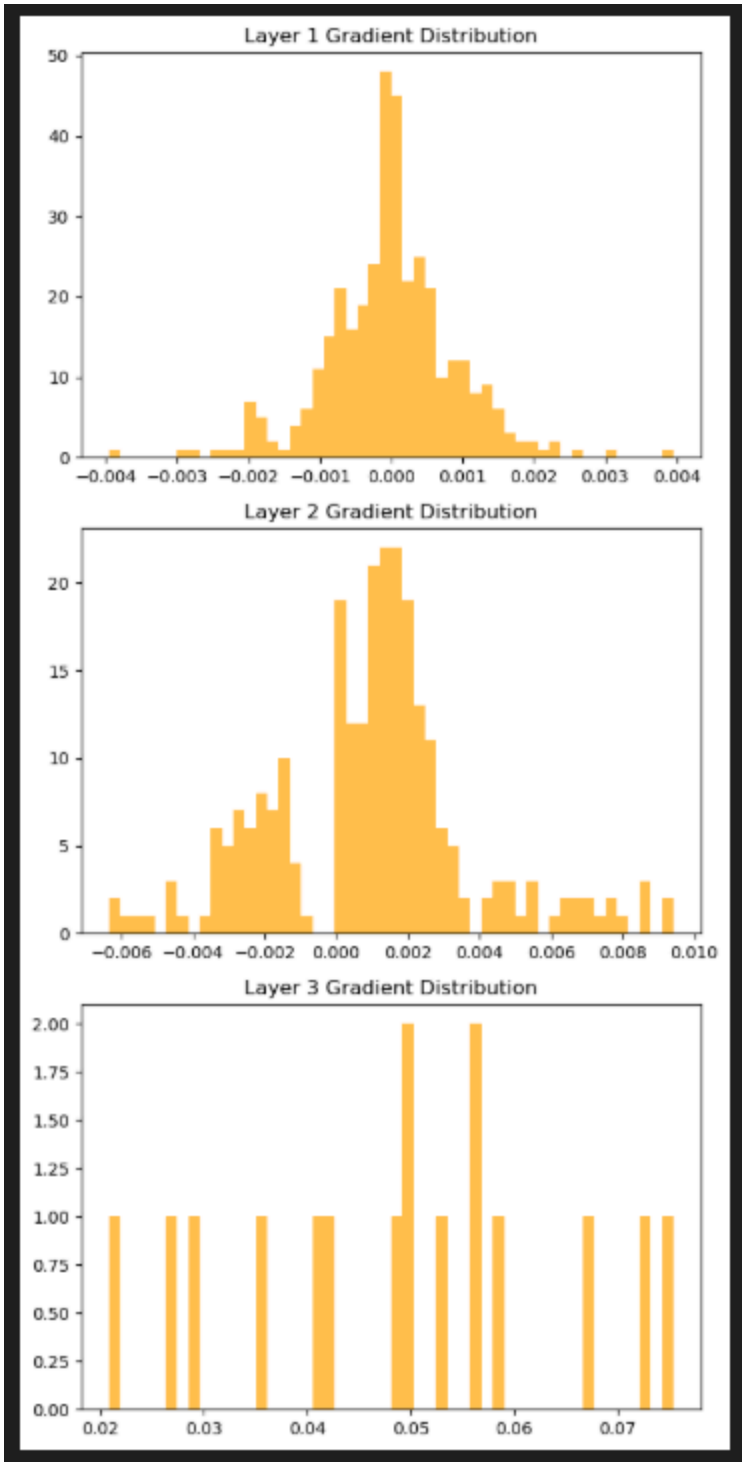

Layer 1 Gradient Distribution
35
30
25
20
15
10
5
0
−0.08
−0.06
−0.04
−0.02
0.00
0.02
0.04
0.06
Layer 2 Gradient Distribution
70
60
50
40
30
20
10
0
−0.02
0.00
0.02
0.04
0.06
0.08
0.10
Layer 3 Gradient Distribution
2.00
1.75
1.50
1.25
1.00
0.75
0.50
0.25
0.00
−0.02
0.00
0.02
0.04
0.06
0.08
0.10

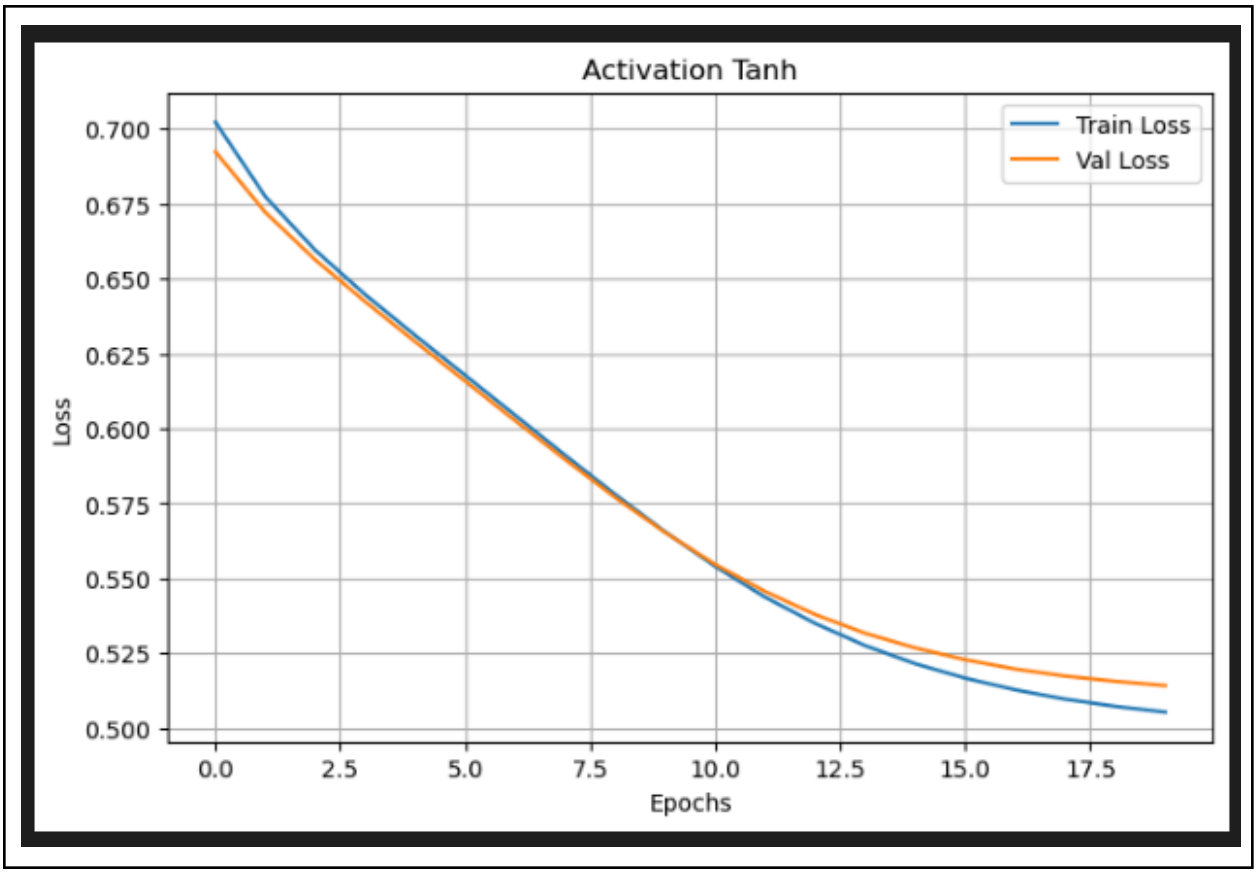



```
--- Activation Sigmoid (Validation) ---
Accuracy : 0.6153
F1 Score : 0.7619
Log Loss : 0.6617
----------------------------------------
```

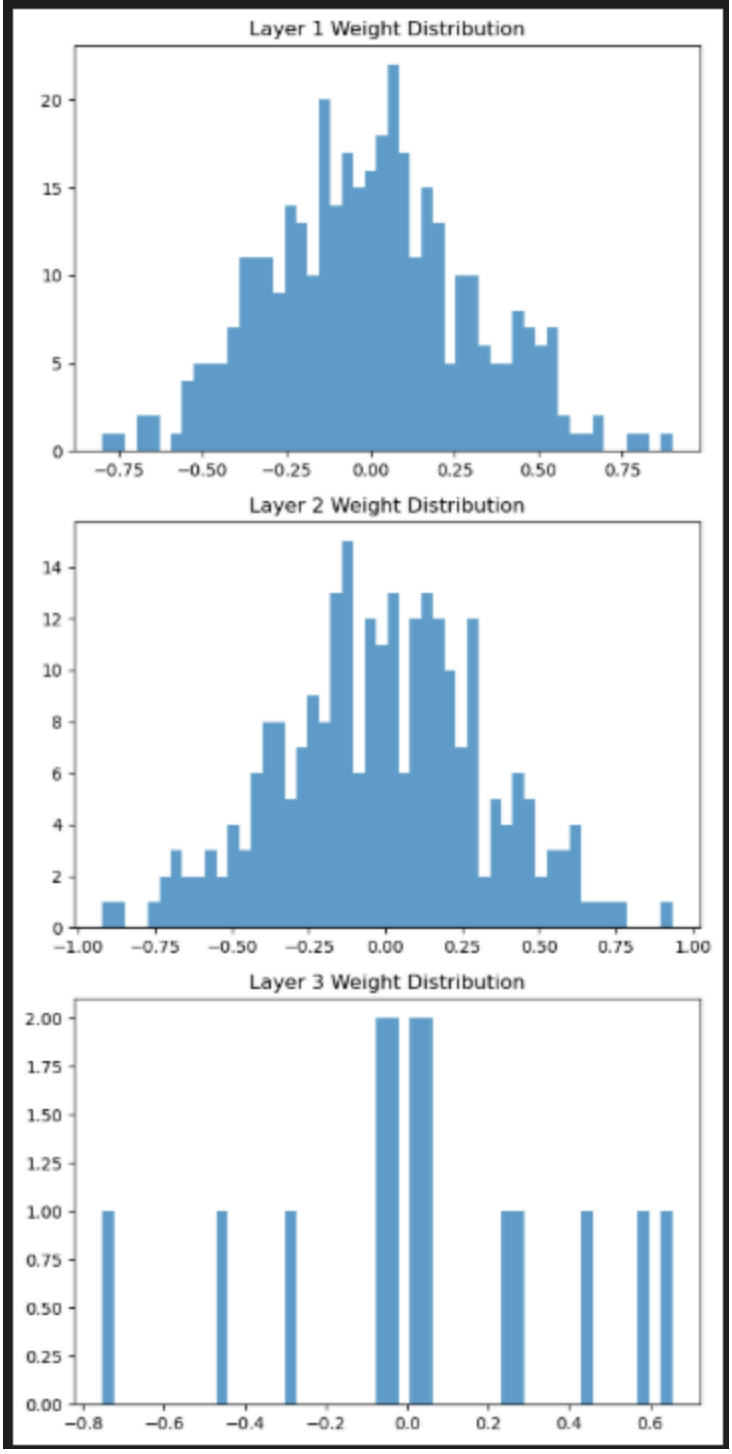
```
Plotting Weights and Gradients distribution (Sigmoid):
```

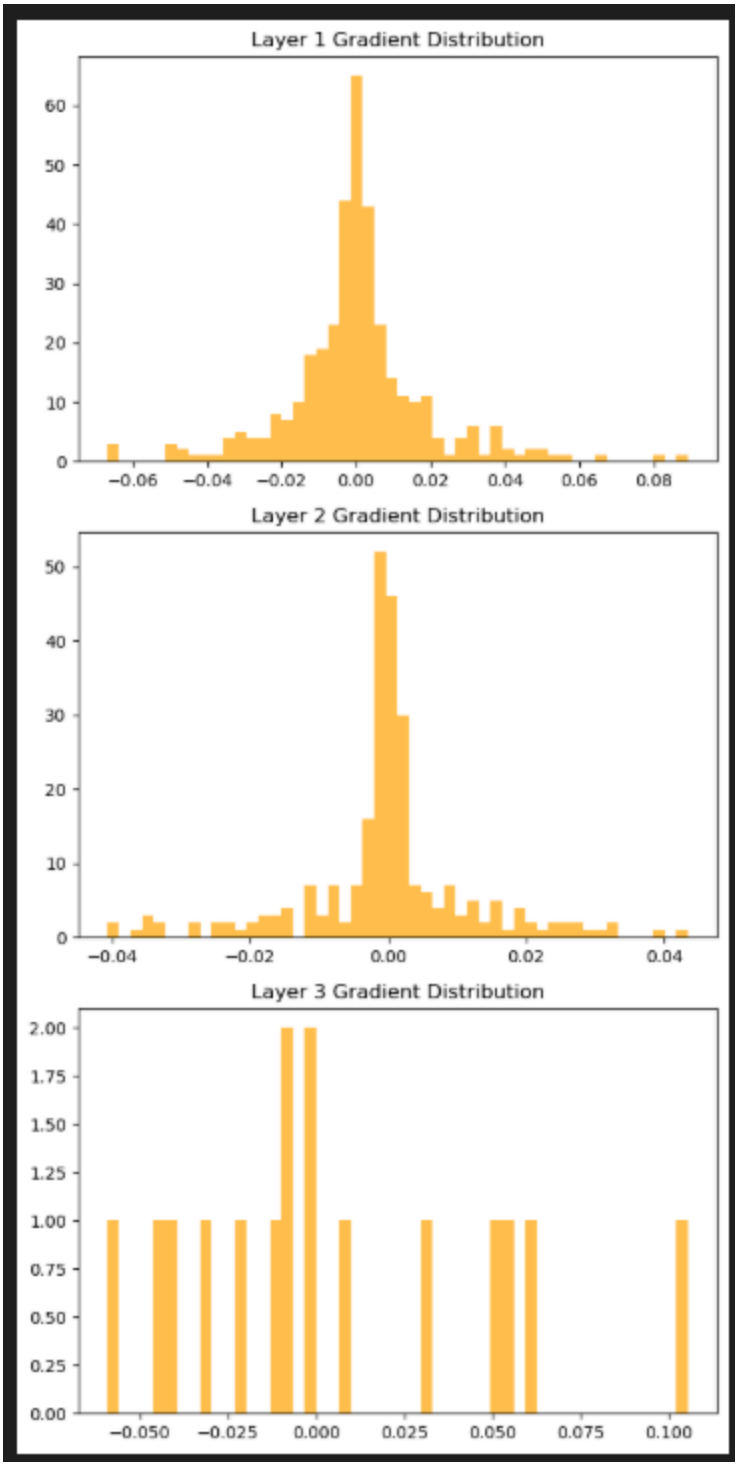


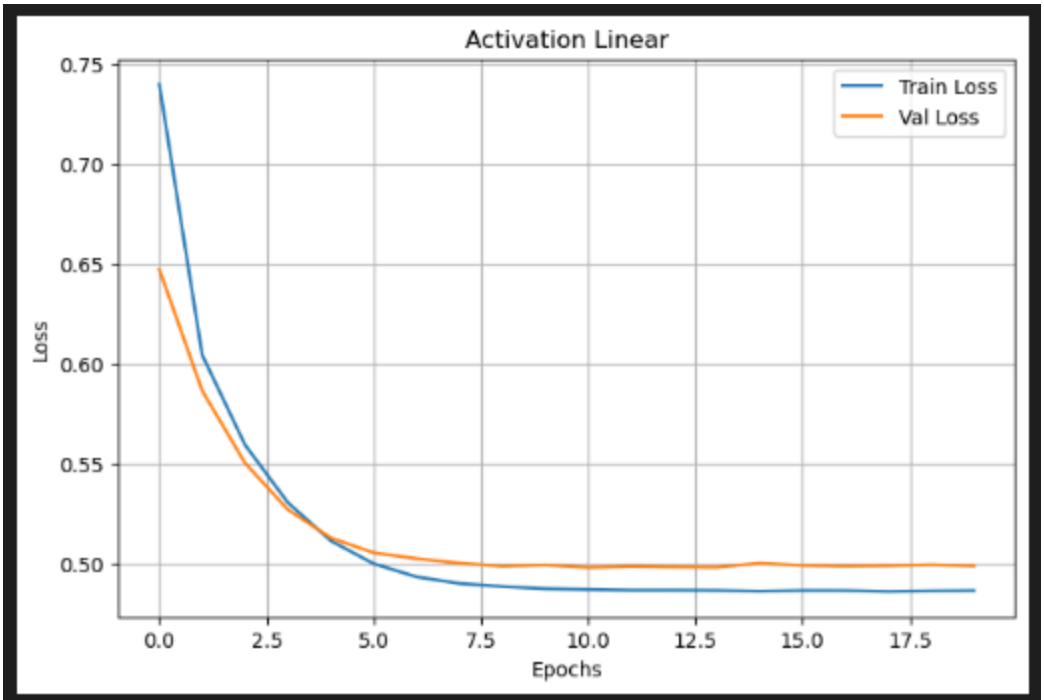

Layer 1 Gradient Distribution
50
40
30
20
10
0
−0.004 −0.003 −0.002 −0.001 0.000 0.001 0.002 0.003 0.004
Layer 2 Gradient Distribution
20
15
10
5
0
−0.006 −0.004 −0.002 0.000 0.002 0.004 0.006 0.008 0.010
Layer 3 Gradient Distribution
2.00
1.75
1.50
1.25
1.00
0.75
0.50
0.25
0.00
0.02 0.03 0.04 0.05 0.06 0.07


Activation Tanh
Train Loss
Val Loss
Loss
0.700
0.675
0.650
0.625
0.600
0.575
0.550
0.525
0.500
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
Epochs

```
Plotting Weights and Gradients distribution (Tanh):
```

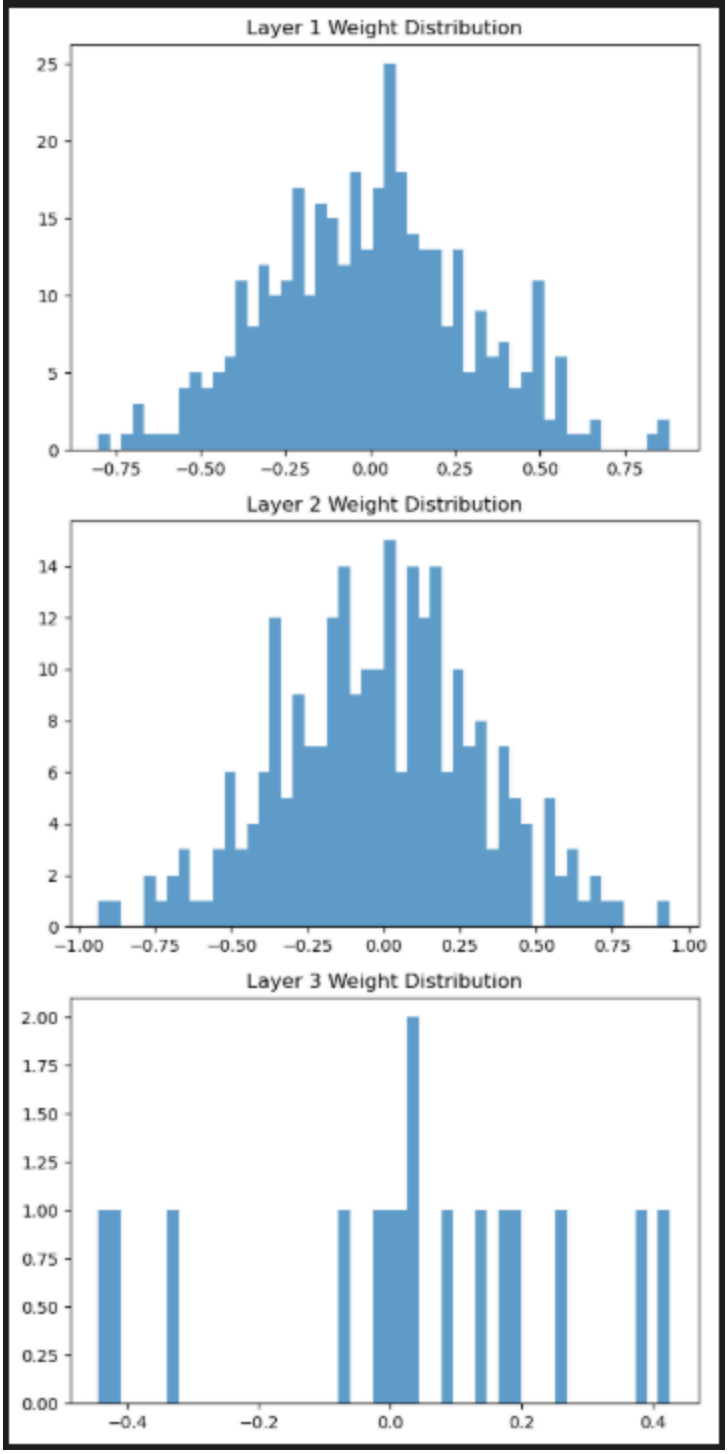


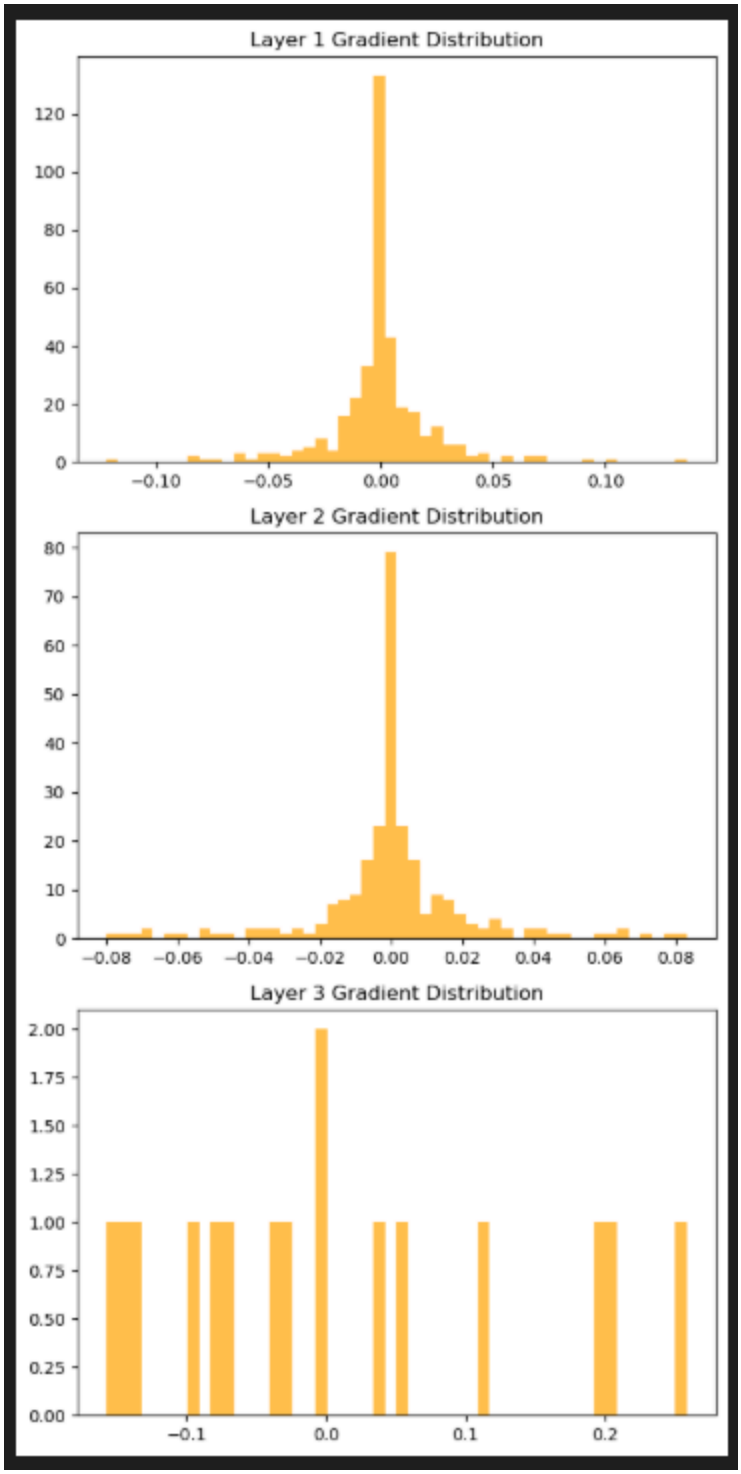

Layer 1 Gradient Distribution
60
50
40
30
20
10
0
−0.06
−0.04
−0.02
0.00
0.02
0.04
0.06
0.08
Layer 2 Gradient Distribution
50
40
30
20
10
0
−0.04
−0.02
0.00
0.02
0.04
Layer 3 Gradient Distribution
2.00
1.75
1.50
1.25
1.00
0.75
0.50
0.25
0.00
−0.050
−0.025
0.000
0.025
0.050
0.075
0.100

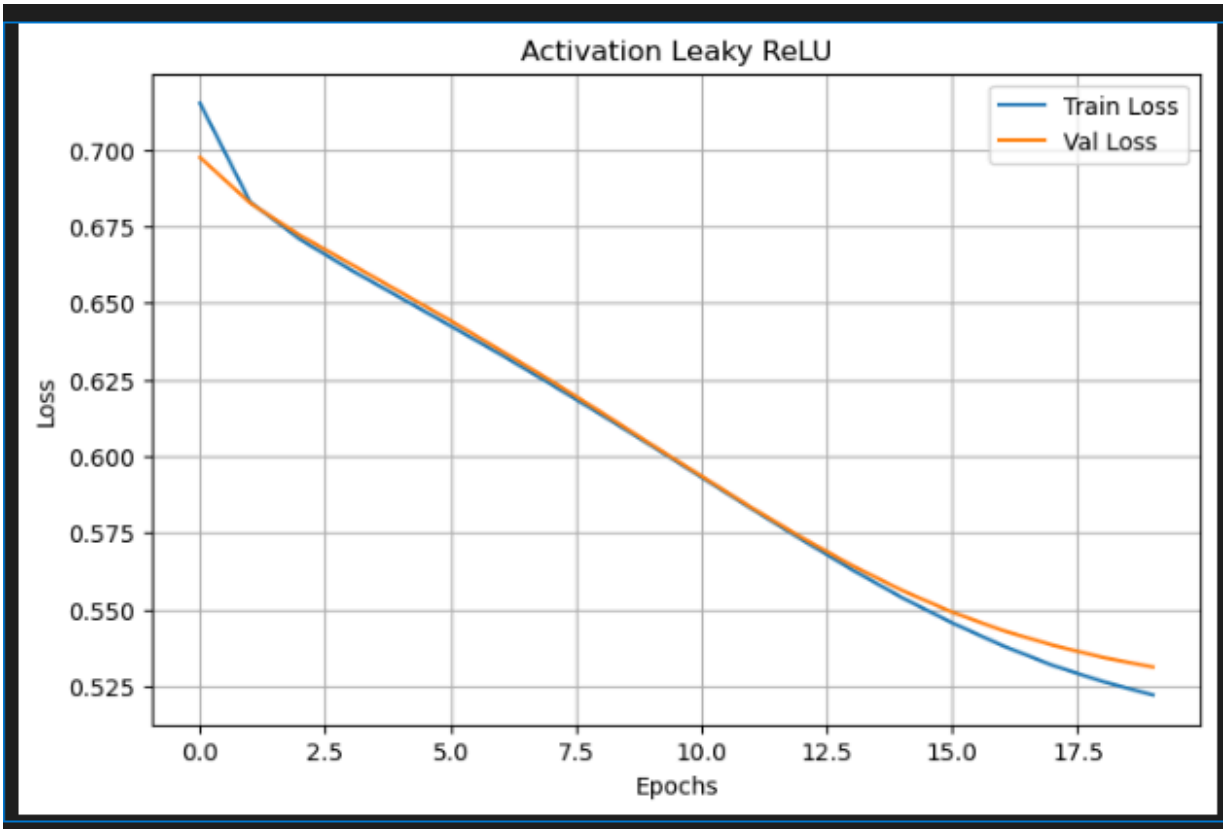



```
--- Activation Linear (Validation) ---
Accuracy : 0.7520
F1 Score : 0.8060
Log Loss : 0.4990
-------------------------------------
```

```
Plotting Weights and Gradients distribution (Linear):
```

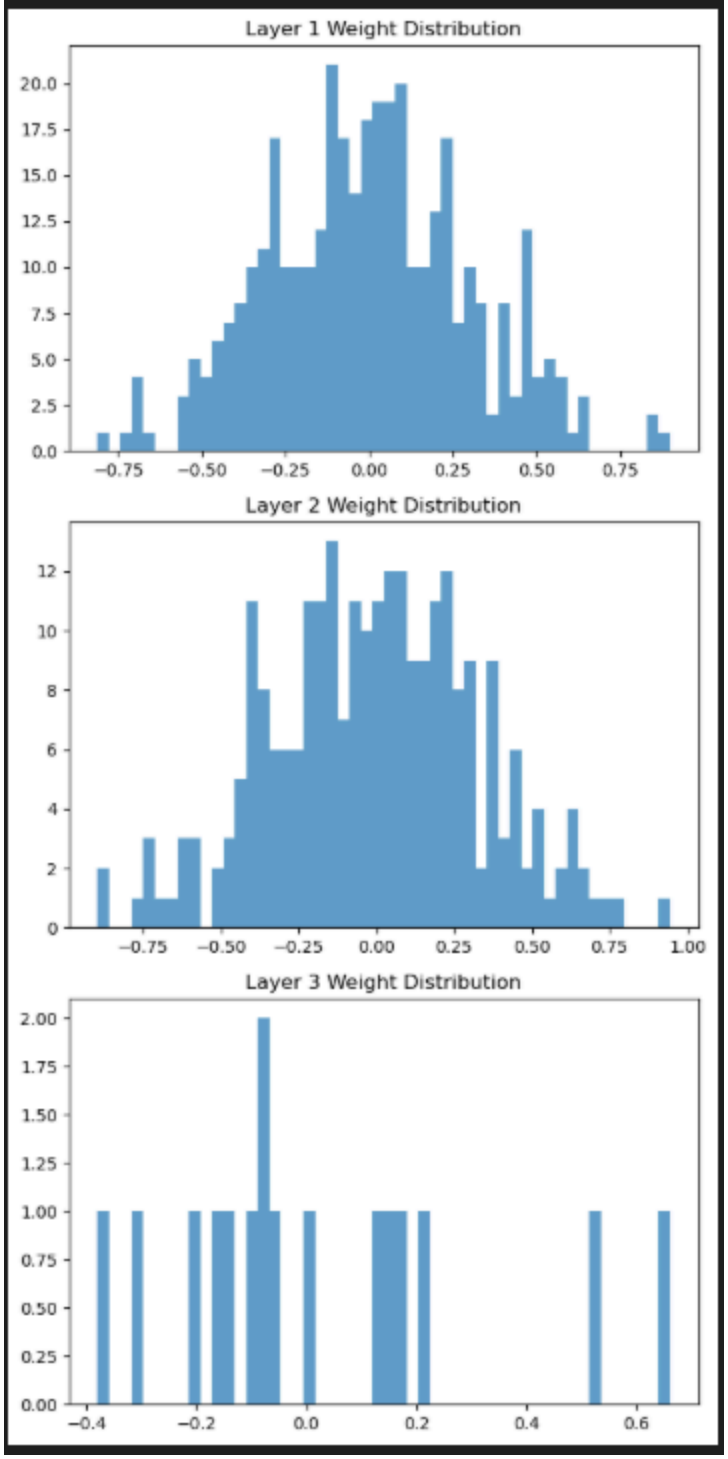


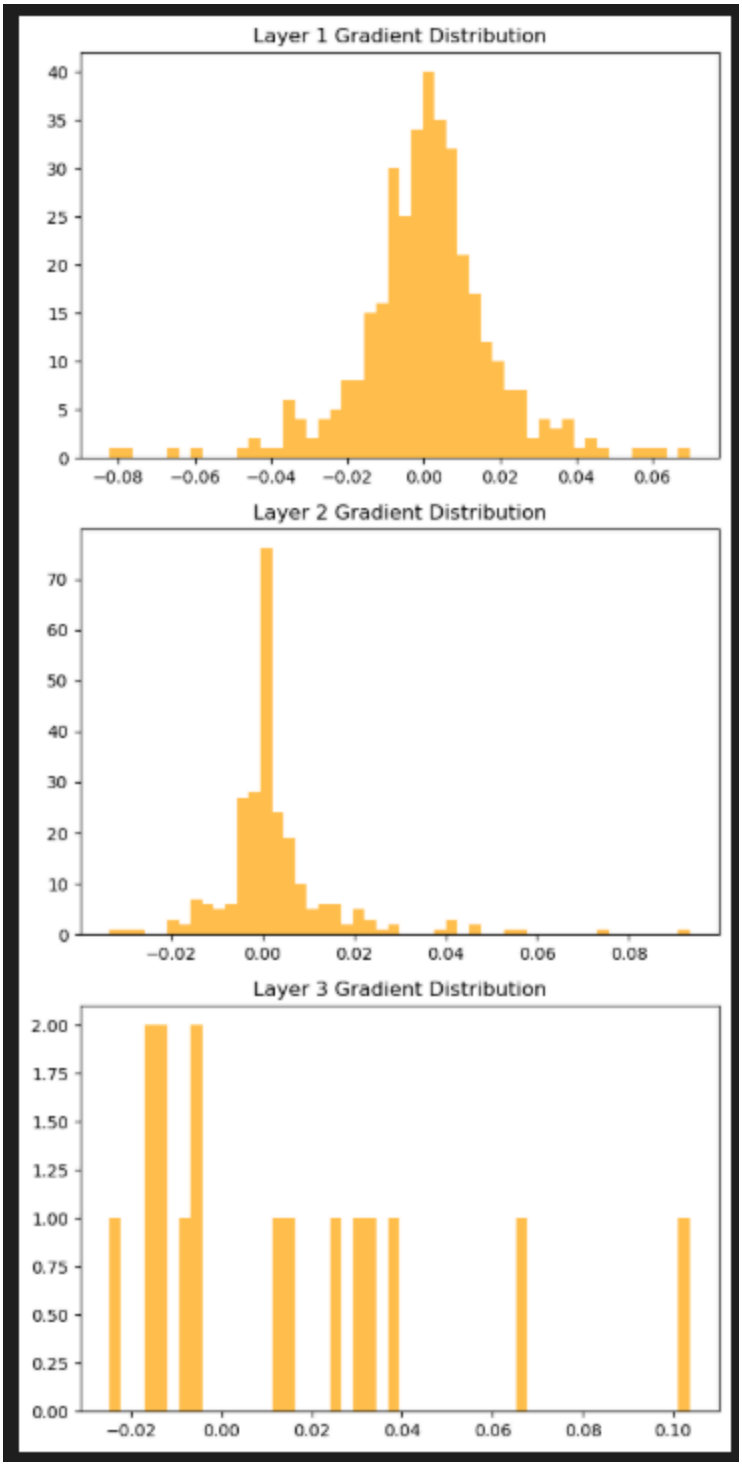

Layer 1 Gradient Distribution
120
100
80
60
40
20
0
−0.10
−0.05
0.00
0.05
0.10
Layer 2 Gradient Distribution
80
70
60
50
40
30
20
10
0
−0.08
−0.06
−0.04
−0.02
0.00
0.02
0.04
0.06
0.08
Layer 3 Gradient Distribution
2.00
1.75
1.50
1.25
1.00
0.75
0.50
0.25
0.00
−0.1
0.0
0.1
0.2

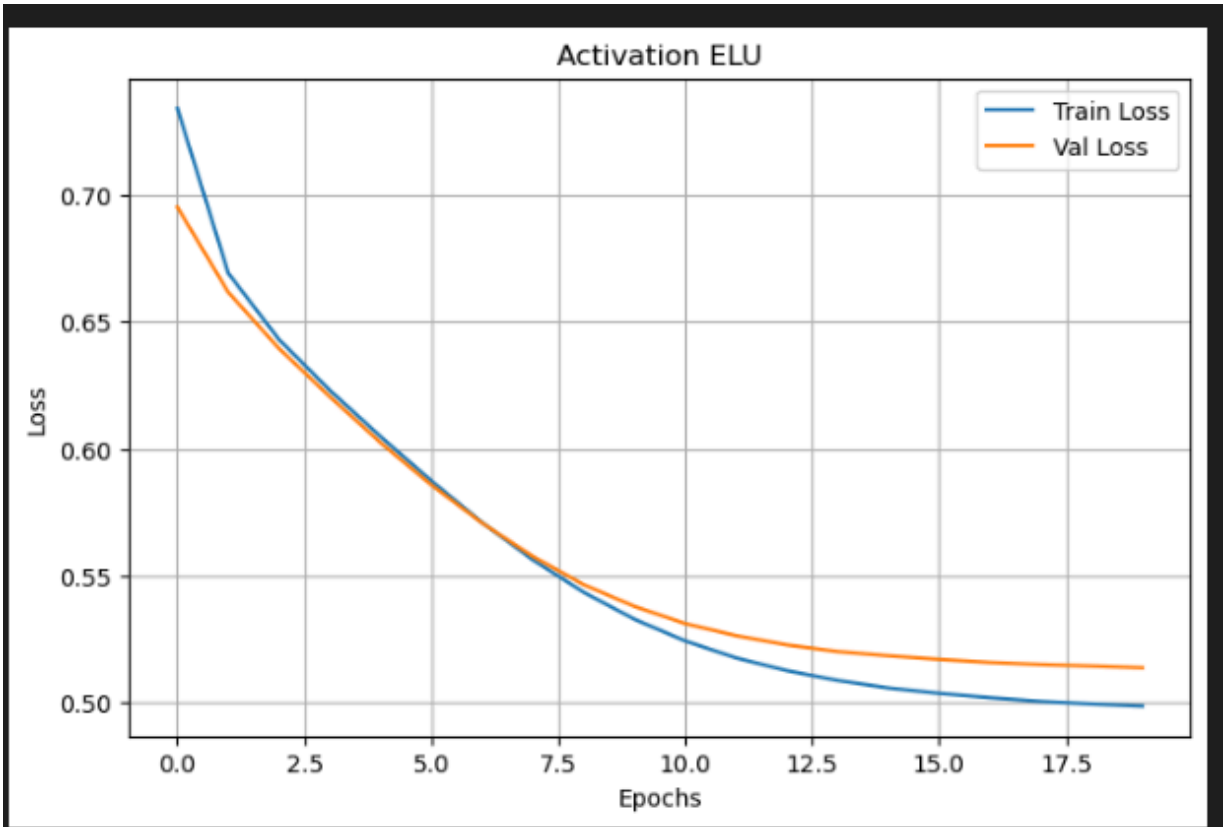



```
--- Activation Leaky ReLU (Validation) ---
Accuracy : 0.7360
F1 Score : 0.7931
Log Loss : 0.5311
-----------------------------------------
Plotting Weights and Gradients distribution (Leaky ReLU):
```

```
Plotting Weights and Gradients distribution (Leaky ReLU):
```

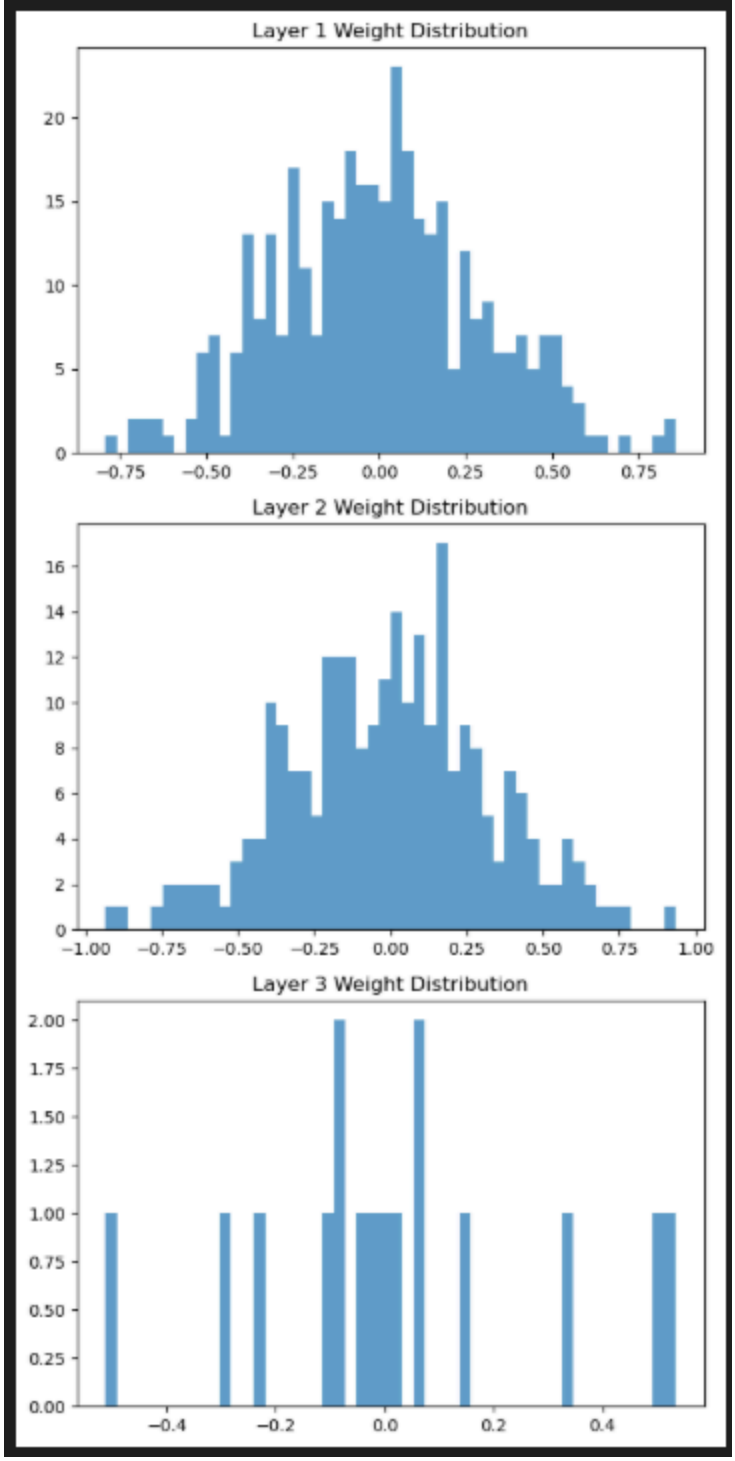


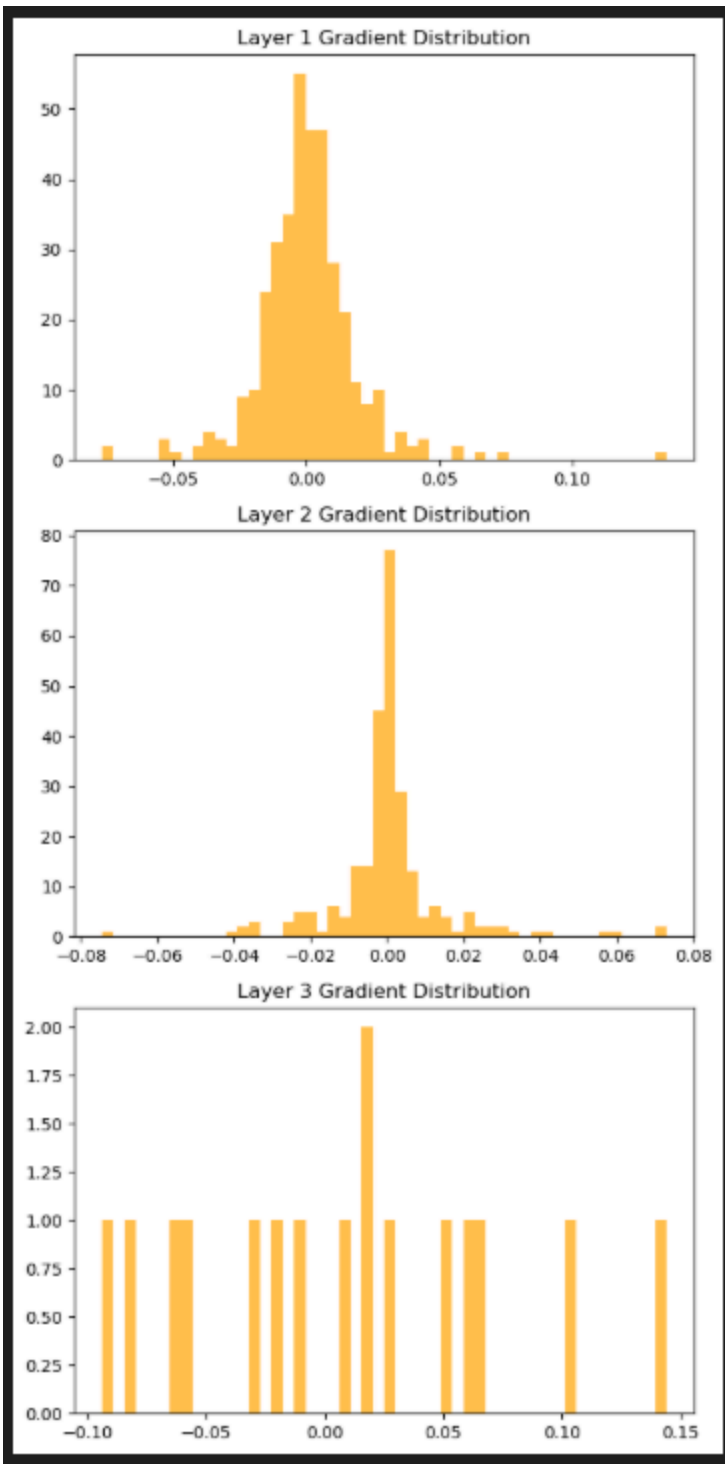

Layer 1 Gradient Distribution
40
35
30
25
20
15
10
5
0
−0.08
−0.06
−0.04
−0.02
0.00
0.02
0.04
0.06
Layer 2 Gradient Distribution
70
60
50
40
30
20
10
0
−0.02
0.00
0.02
0.04
0.06
0.08
Layer 3 Gradient Distribution
2.00
1.75
1.50
1.25
1.00
0.75
0.50
0.25
0.00
−0.02
0.00
0.02
0.04
0.06
0.08
0.10




```
--- Activation ELU (Validation) ---
Accuracy : 0.7460
F1 Score : 0.7998
Log Loss : 0.5137
-----------------------------------
```

```
Plotting Weights and Gradients distribution (ELU):
```




Layer 1 Gradient Distribution
50
40
30
20
10
0
−0.05
0.00
0.05
0.10
Layer 2 Gradient Distribution
80
70
60
50
40
30
20
10
0
−0.08
−0.06
−0.04
−0.02
0.00
0.02
0.04
0.06
0.08
Layer 3 Gradient Distribution
2.00
1.75
1.50
1.25
1.00
0.75
0.50
0.25
0.00
−0.10
−0.05
0.00
0.05
0.10
0.15

Aktivasi	Accuracy	F1-score	Log Loss
ReLU	0.7367	0.7937	0.5317
Sigmoid	0.6153	0.7619	0.6617
Tanh	0.7507	0.8070	0.5142
Linear	0.7520	0.8060	0.4990
Leaky ReLU	0.7360	0.7931	0.5311
ELU	0.7460	0.7998	0.5137

Analisis

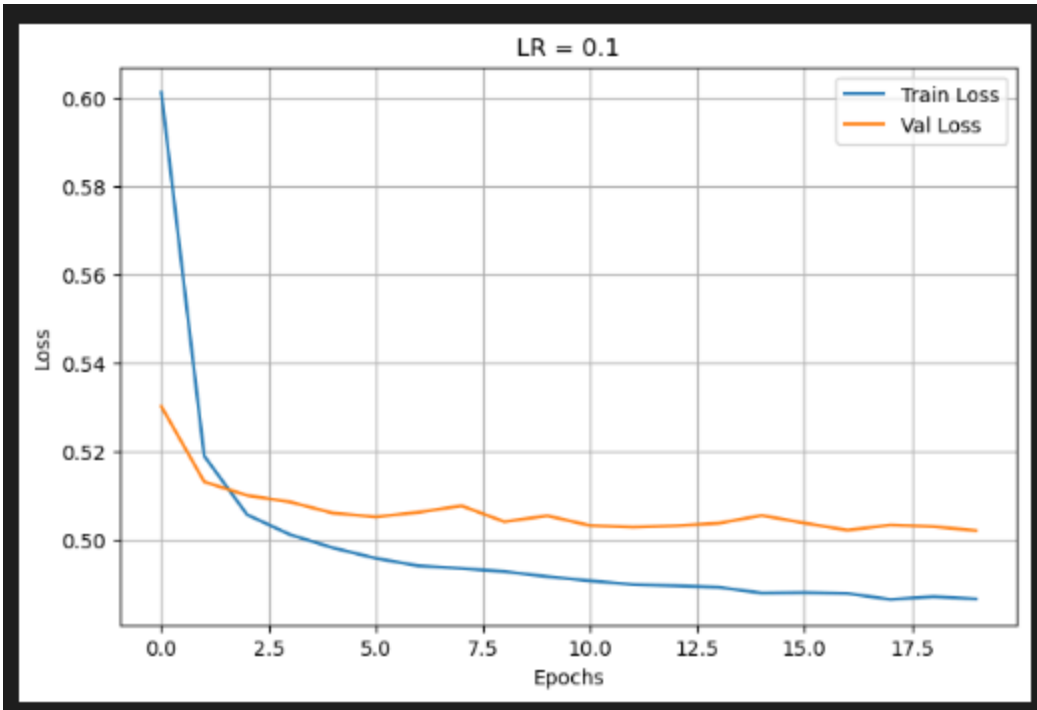
Hasil percobaan menunjukkan bahwa fungsi aktivasi berpengaruh terhadap kecepatan konvergensi dan performa akhir model. Aktivasi linear, tanh, dan ELU memberikan hasil yang relatif lebih baik, sedangkan sigmoid menunjukkan performa paling lemah karena penurunan loss berlangsung lebih lambat dan nilai akhirnya lebih tinggi. Hal ini menunjukkan bahwa kemampuan fungsi aktivasi dalam menjaga aliran gradien sangat memengaruhi efektivitas pembelajaran. Pada sigmoid, gradien cenderung kecil sehingga pembaruan bobot menjadi kurang optimal, sedangkan pada linear proses optimasi terlihat lebih stabil. Secara keseluruhan, pemilihan fungsi aktivasi yang tepat dapat meningkatkan performa model secara signifikan.

3.3 Pengaruh Learning Rate

Informasi

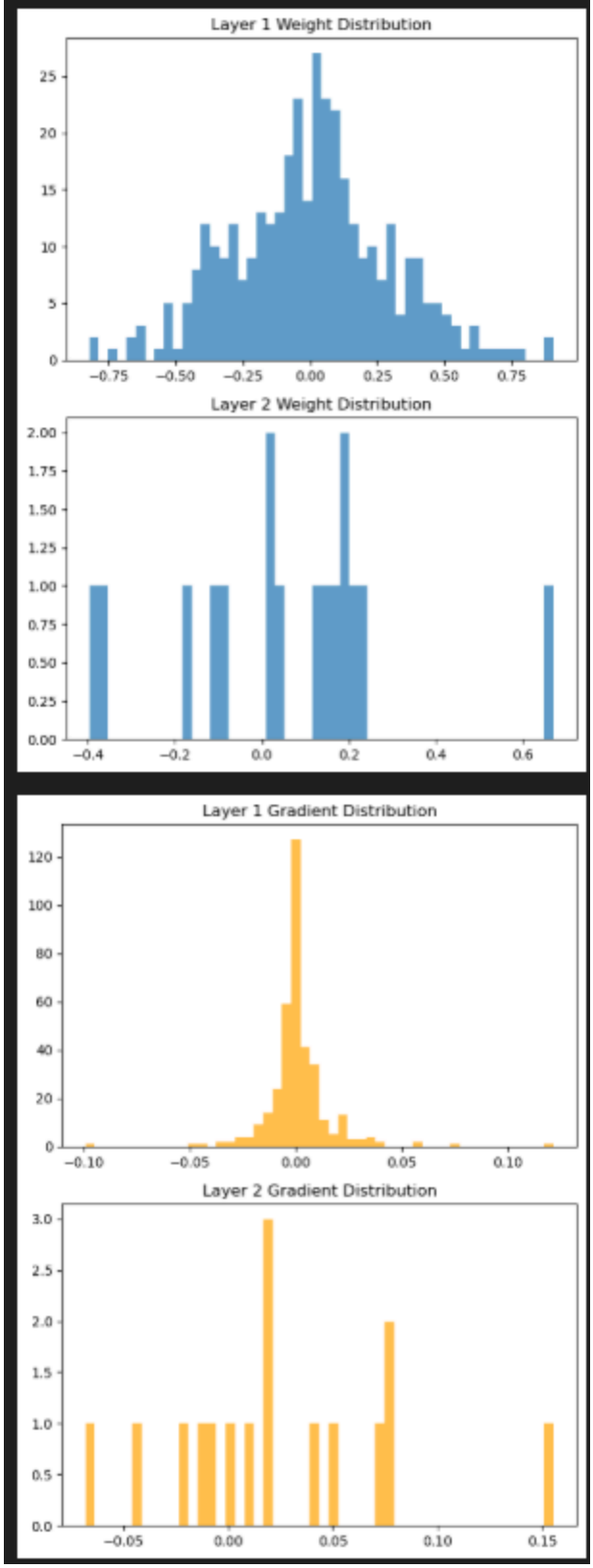
Learning rate diuji pada:
- 0.1
- 0.01
- 0.001

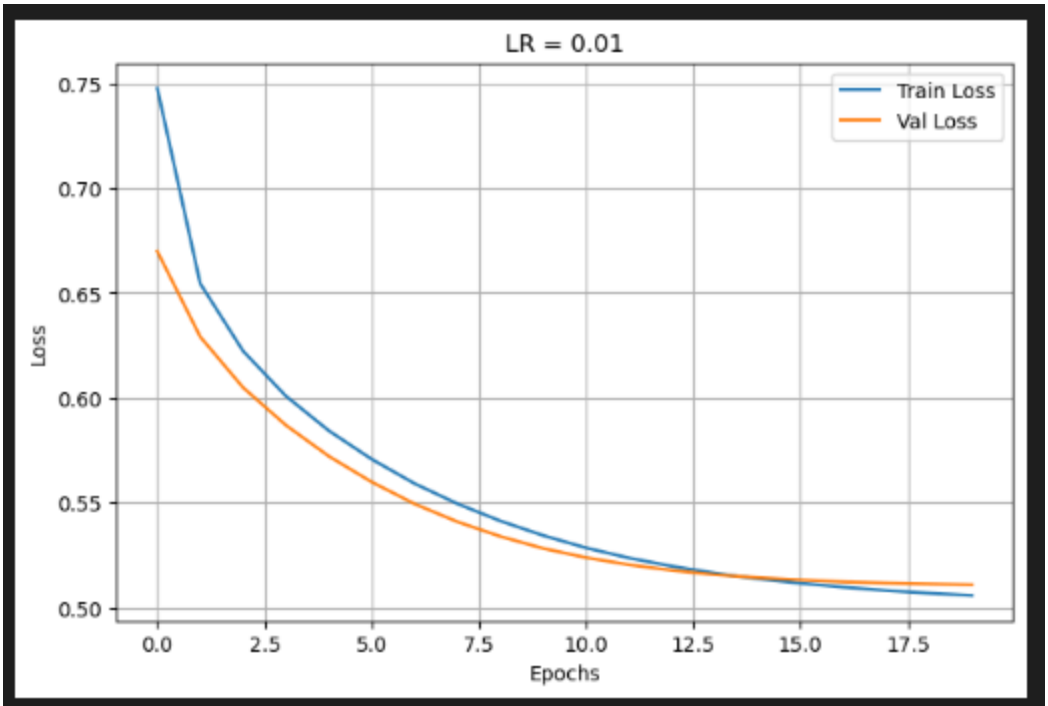
Grafik




```
--- Learning Rate 0.1 (Validation) ---
Accuracy : 0.7587
F1 Score : 0.8132
Log Loss : 0.5020
--------------------------------------
```

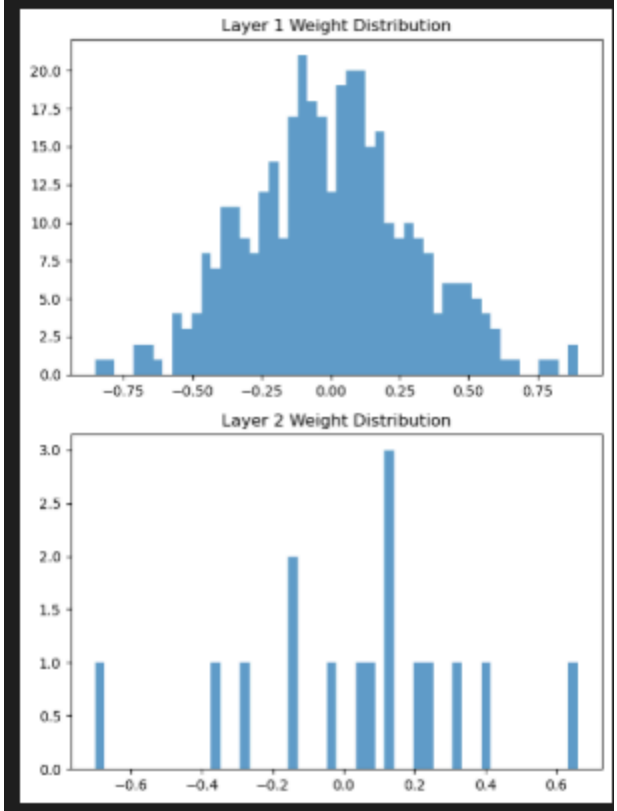
```
Plotting Weights and Gradients distribution (LR = 0.1):
```

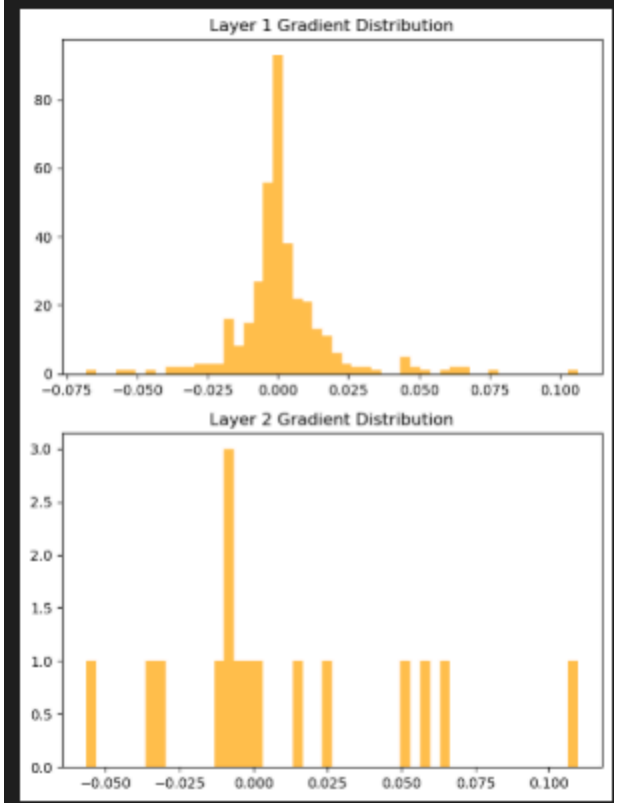


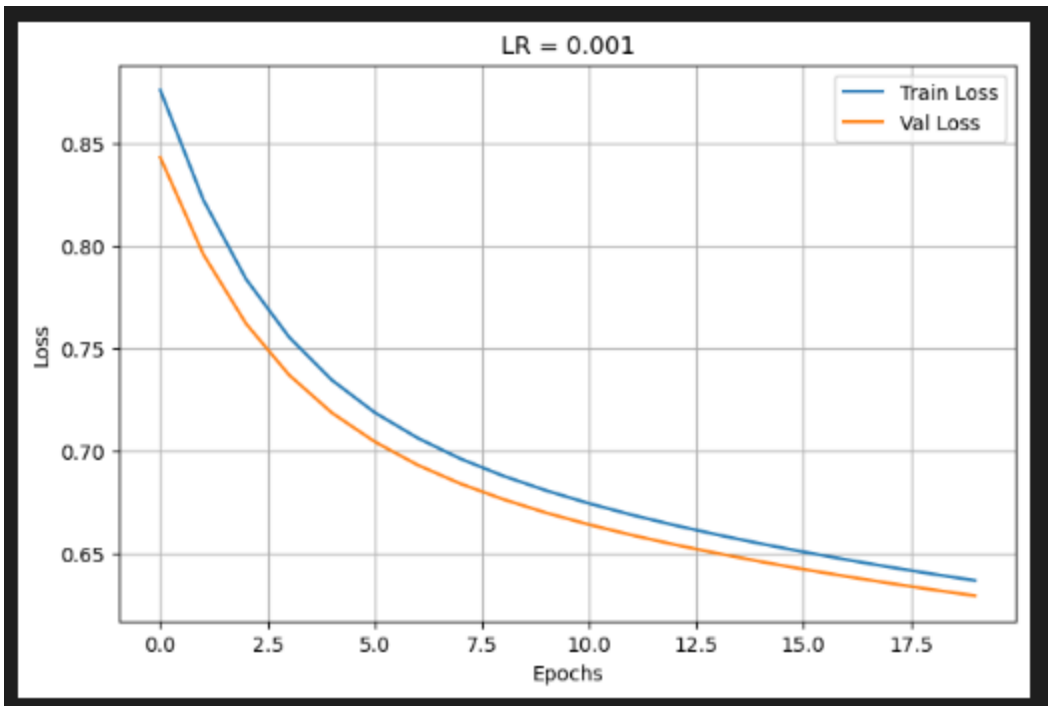



```
--- Learning Rate 0.01 (Validation) ---
Accuracy : 0.7540
F1 Score : 0.8083
Log Loss : 0.5110
----------------------------------------
```

```
Plotting Weights and Gradients distribution (LR = 0.01):
```

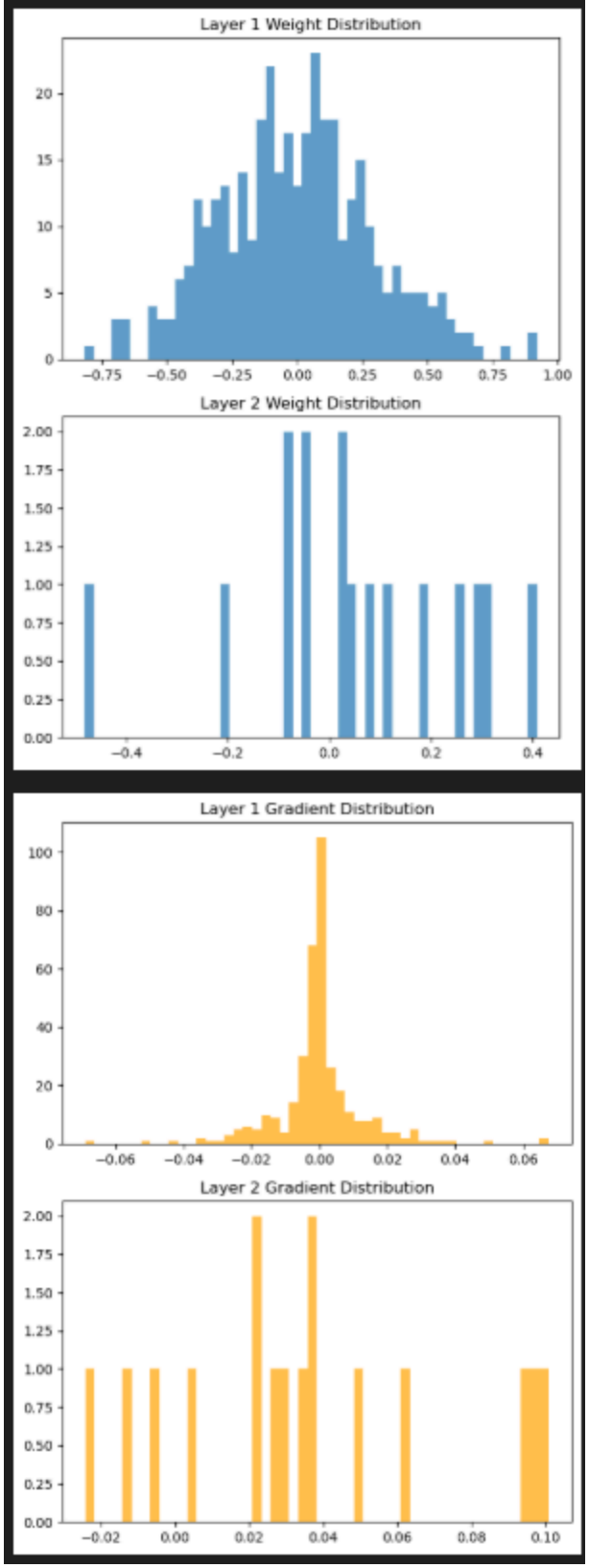







```
--- Learning Rate 0.001 (Validation) ---
Accuracy : 0.6420
F1 Score : 0.7405
Log Loss : 0.6292
----------------------------------------
```

```
Plotting Weights and Gradients distribution (LR = 0.001):
```



Learning Rate	Accuracy	F1-score	Log Loss
0.1	0.7587	0.8132	0.5020
0.01	0.7540	0.8083	0.5110
0.001	0.6420	0.7405	0.6292

Analisis

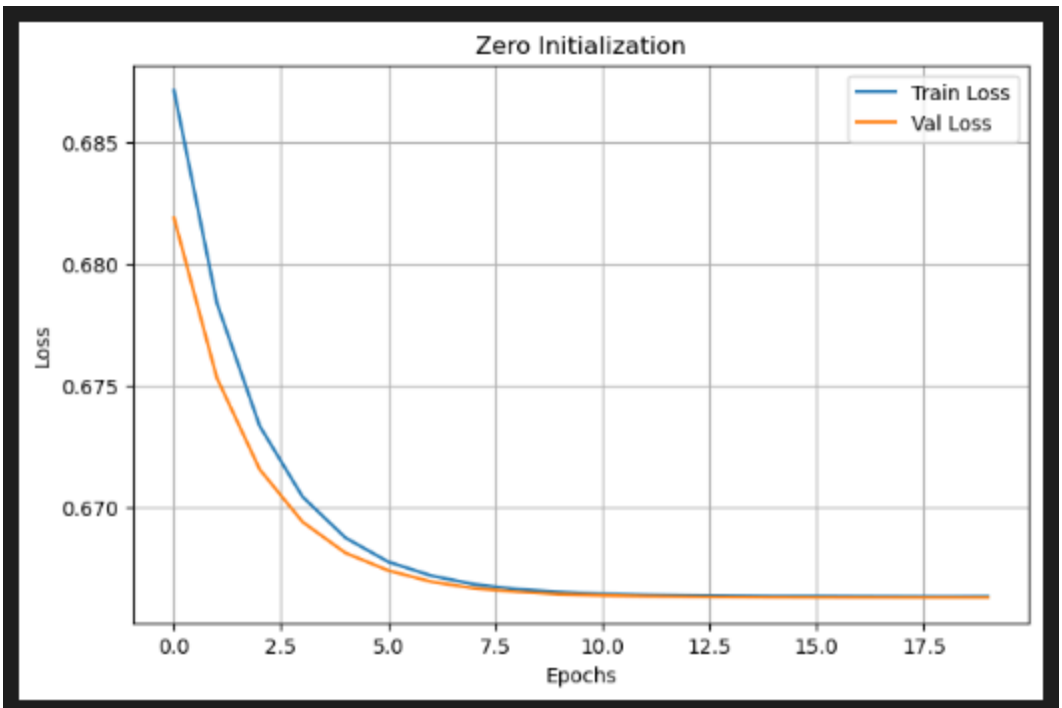
- 0.1 memberi hasil terbaik
- 0.001 terlalu kecil sehingga pembelajaran lambat dan performa buruk
- 0.01 masih stabil, tapi kalah dari 0.1

3.4 Pengaruh Inisialisasi Bobot

Informasi

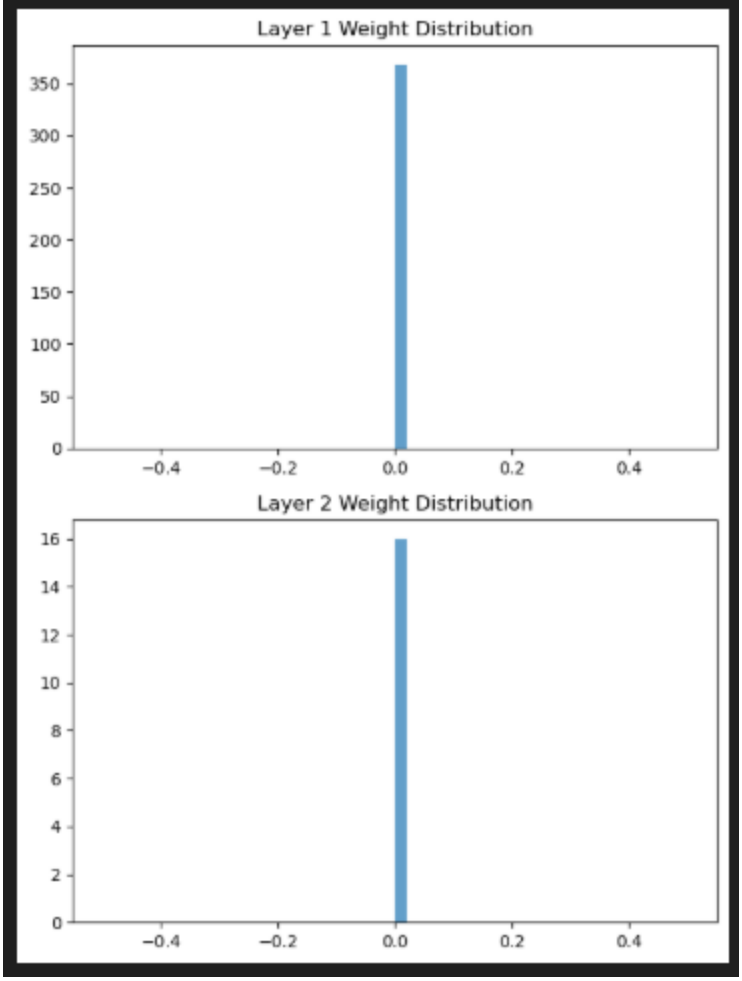
Metode yang diuji:
- Zero
- Random uniform
- Random normal
- Xavier
- He

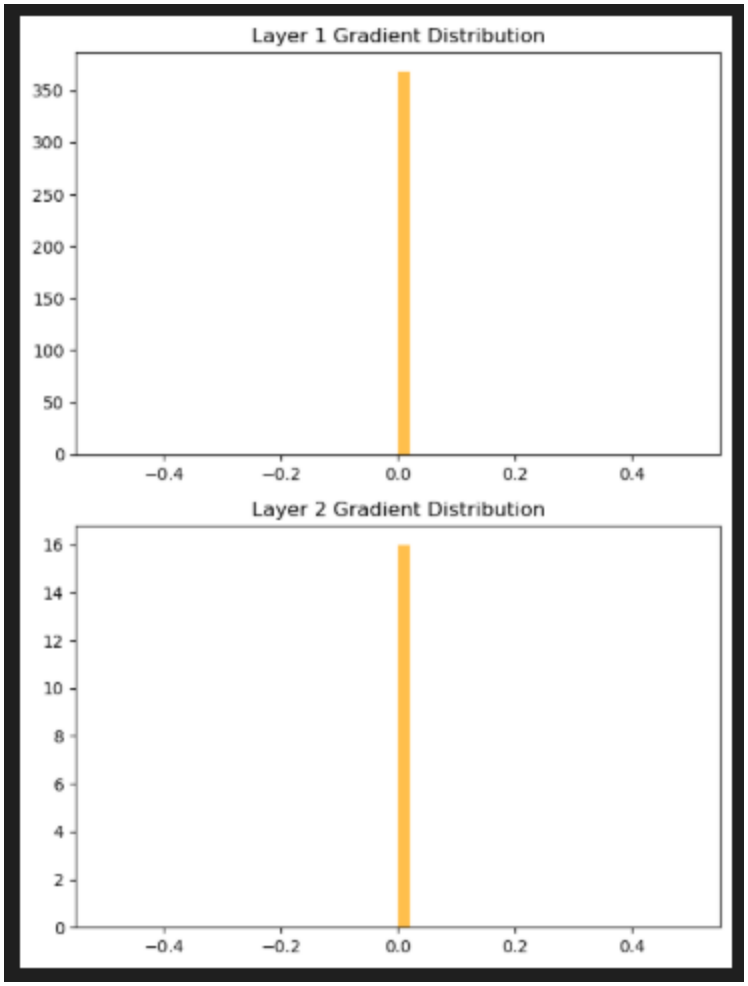
Grafik

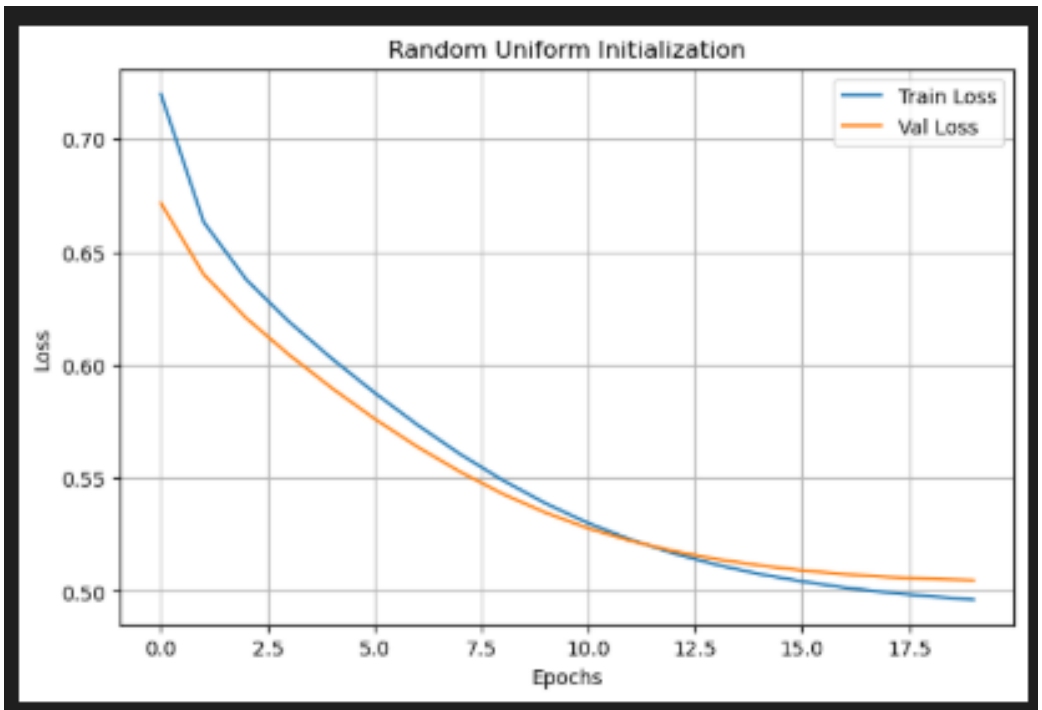



```
--- Zero Initialization (Validation) ---
Accuracy : 0.6153
F1 Score : 0.7619
Log Loss : 0.6663
-----------------------------------------
```

```
Plotting Weights and Gradients distribution (Zero Initialization):
```

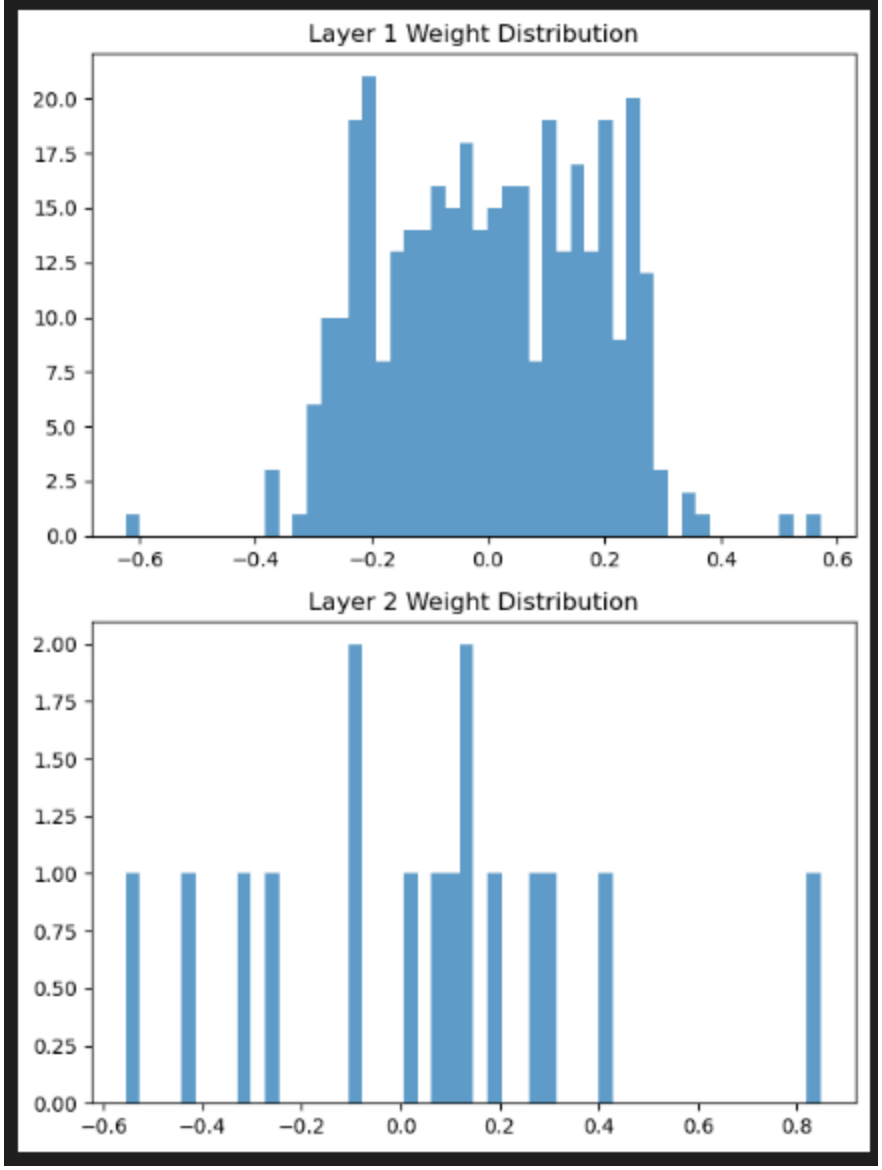


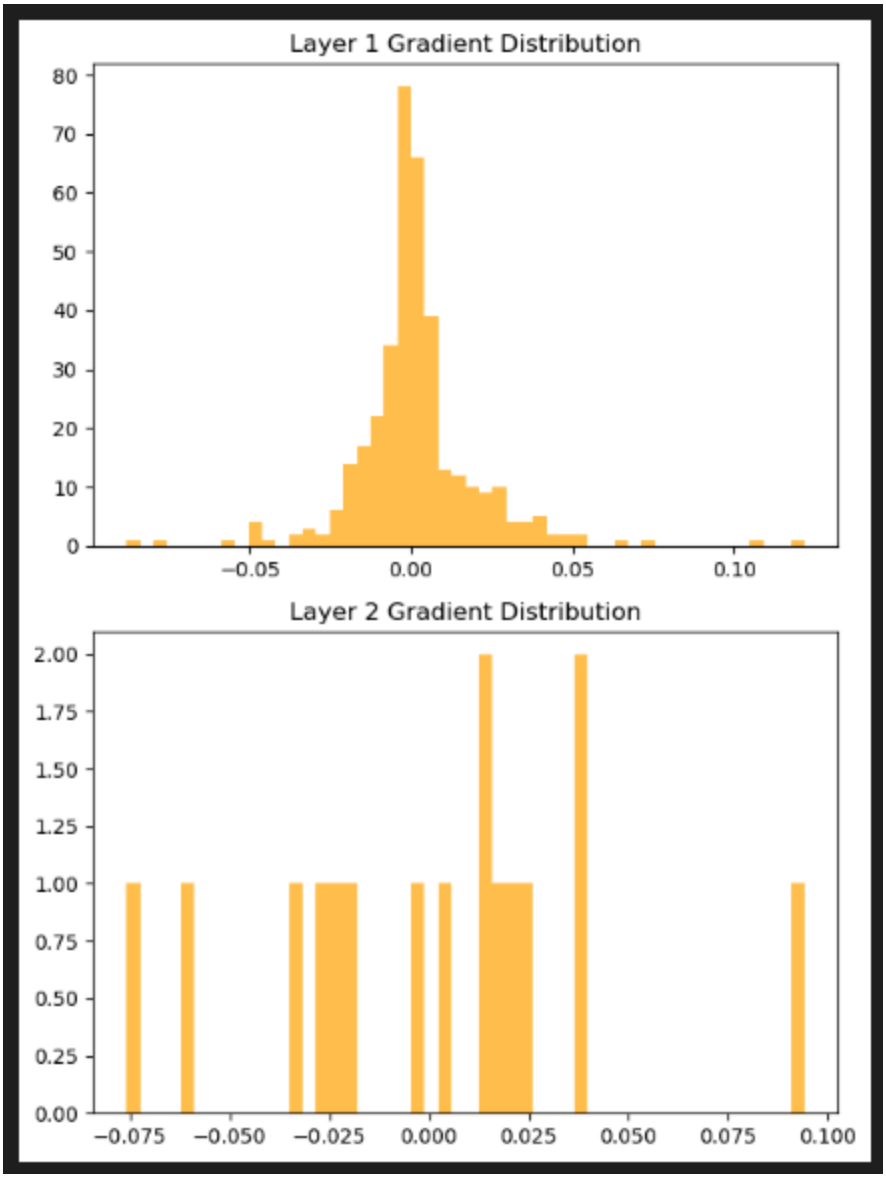

Layer 1 Gradient Distribution
350
300
250
200
150
100
50
0
−0.4
−0.2
0.0
0.2
0.4
Layer 2 Gradient Distribution
16
14
12
10
8
6
4
2
0
−0.4
−0.2
0.0
0.2
0.4

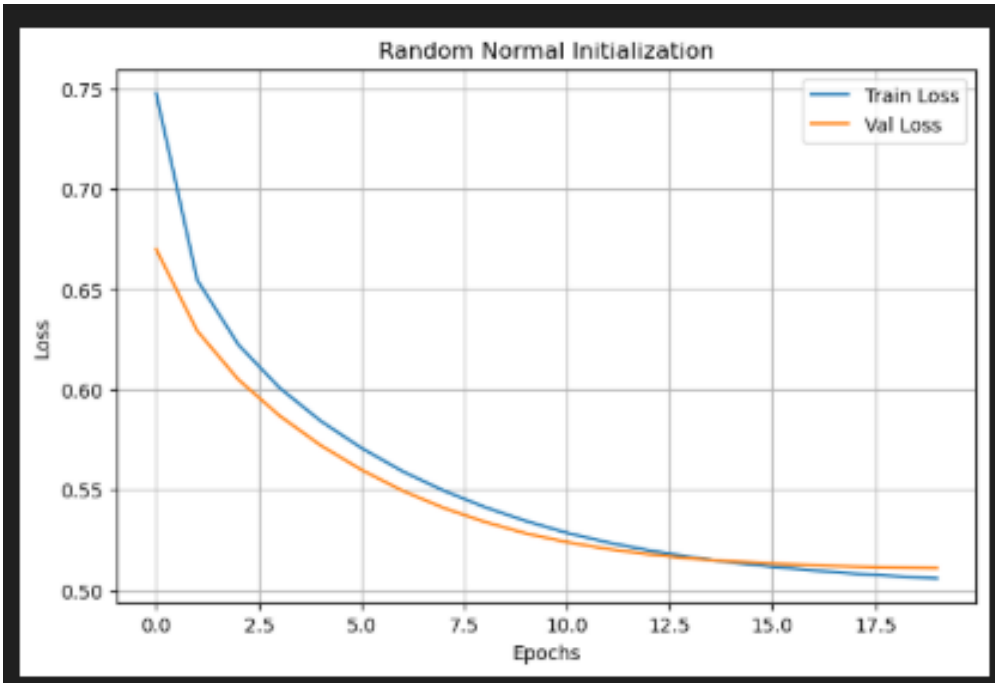



```
--- Random Uniform Initialization (Validation) ---
Accuracy : 0.7600
F1 Score : 0.8115
Log Loss : 0.5049
-------------------------------------------------
```

```
Plotting Weights and Gradients distribution (Random Uniform Initialization):
```

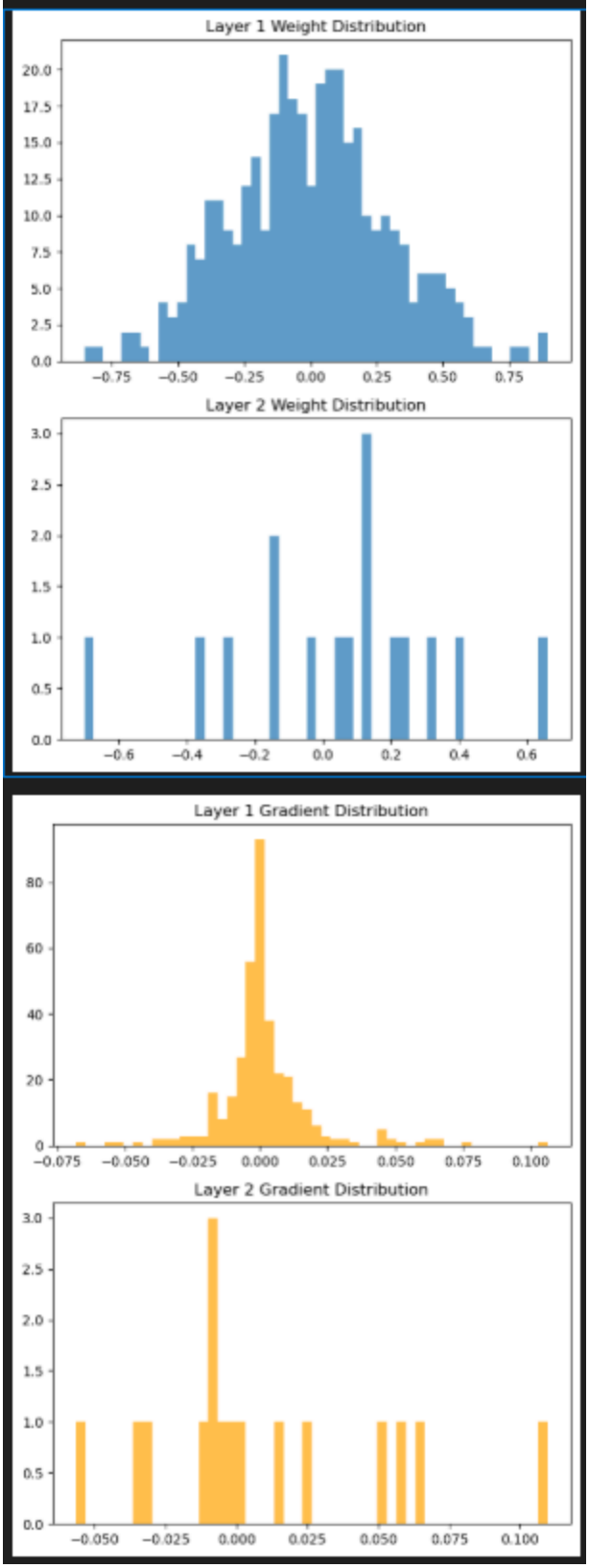


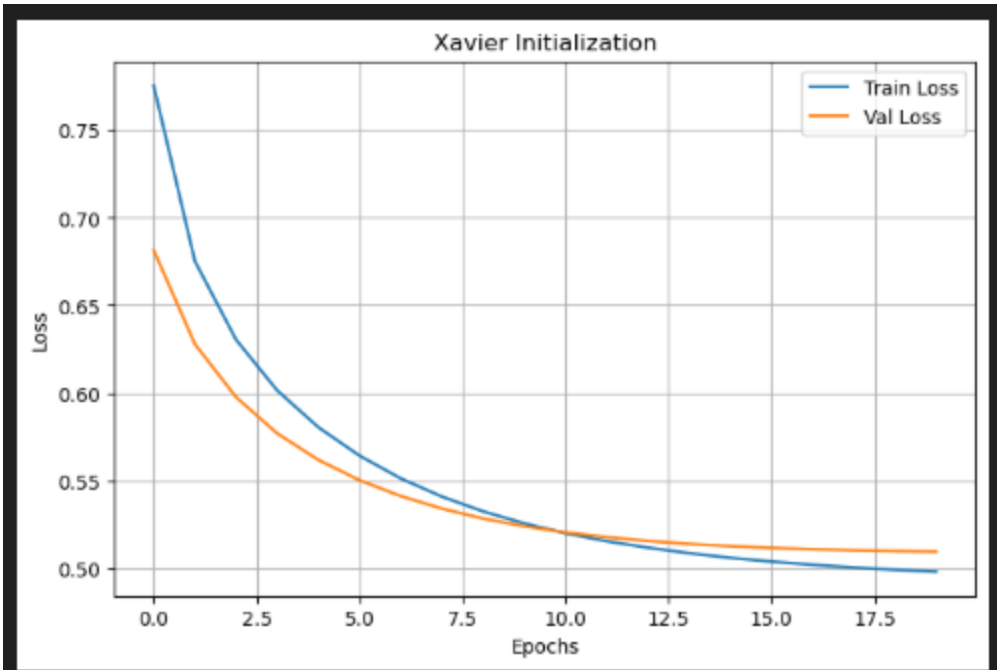

Layer 1 Gradient Distribution
80
70
60
50
40
30
20
10
0
−0.05
0.00
0.05
0.10
Layer 2 Gradient Distribution
2.00
1.75
1.50
1.25
1.00
0.75
0.50
0.25
0.00
−0.075
−0.050
−0.025
0.000
0.025
0.050
0.075
0.100




```
--- Random Normal Initialization (Validation) ---
Accuracy : 0.7540
F1 Score : 0.8083
Log Loss : 0.5110
-----------------------------------------------
```

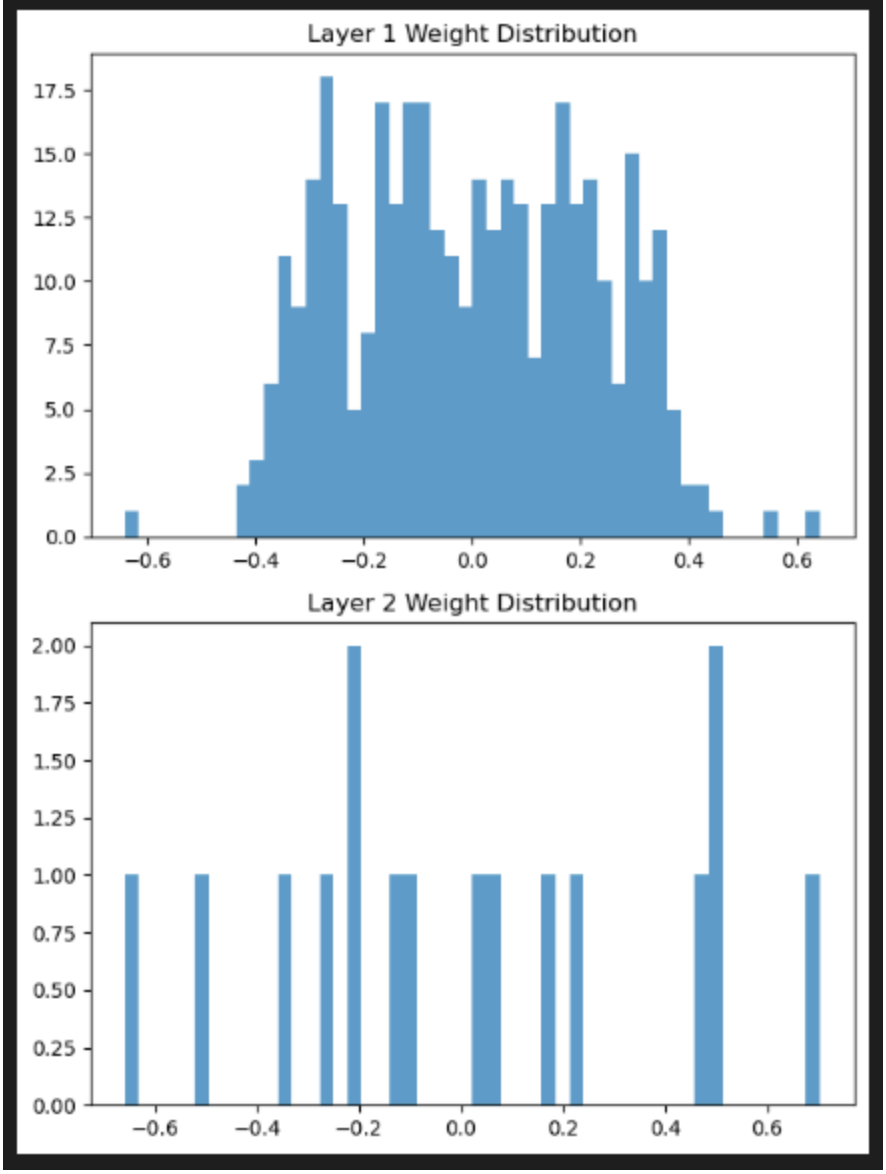
```
Plotting Weights and Gradients distribution (Random Normal Initialization):
```

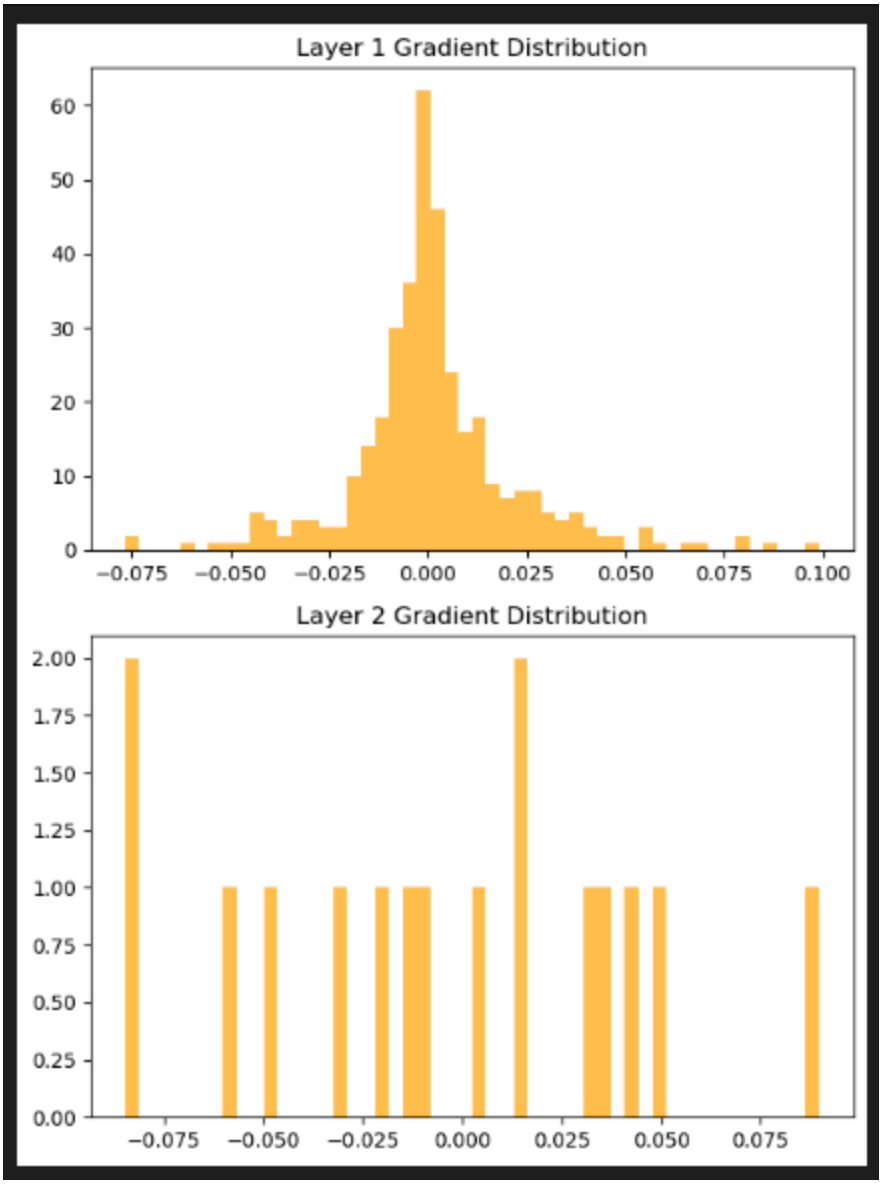


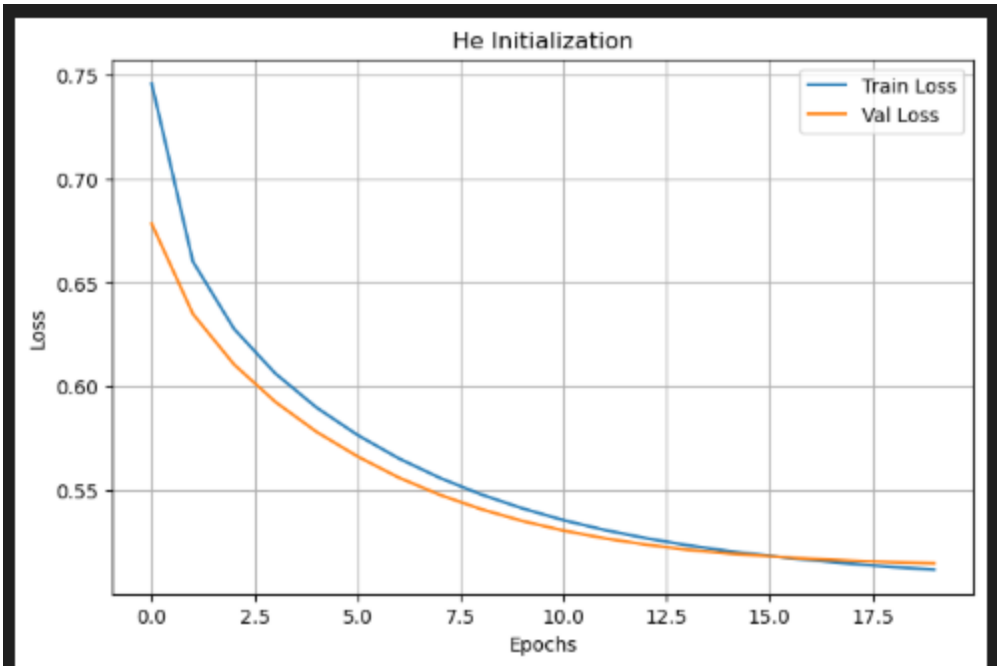



```
--- Xavier Initialization (Validation) ---
Accuracy : 0.7573
F1 Score : 0.8112
Log Loss : 0.5094
----------------------------------------
```

```
Plotting Weights and Gradients distribution (Xavier Initialization):
```

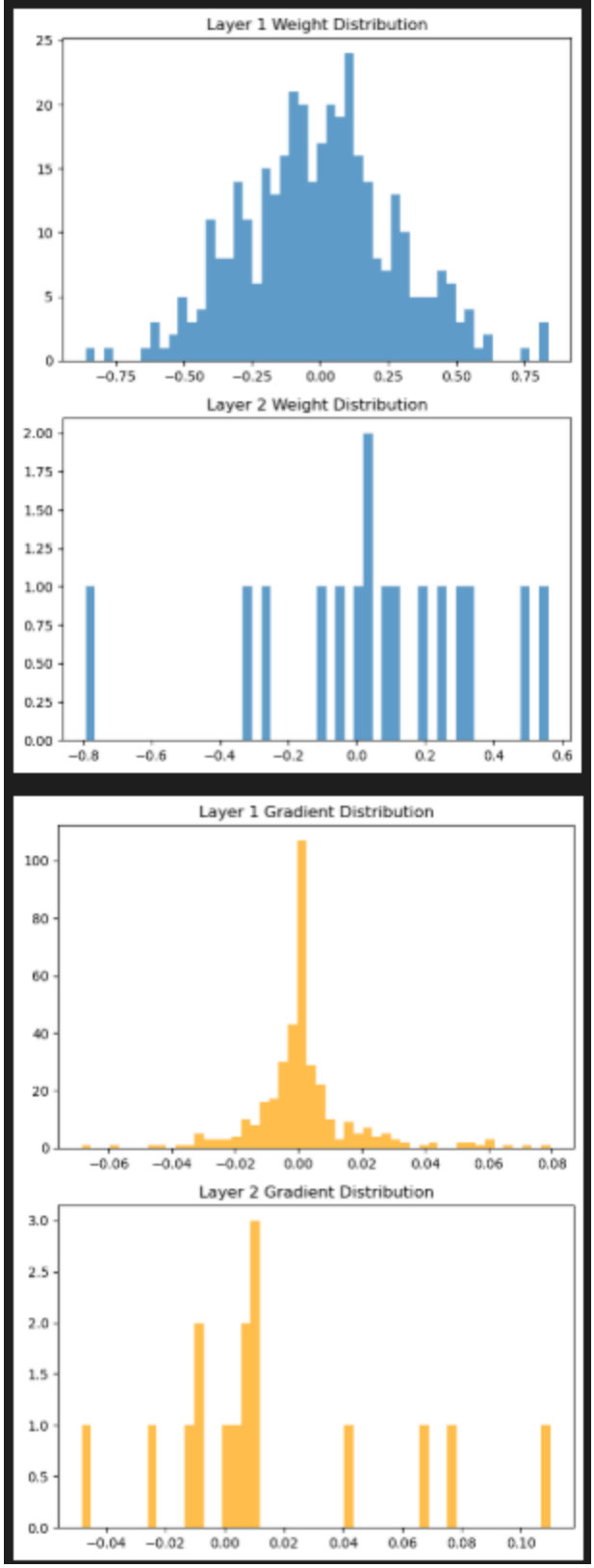



Layer 1 Gradient Distribution
60
50
40
30
20
10
0
−0.075
−0.050
−0.025
0.000
0.025
0.050
0.075
0.100
Layer 2 Gradient Distribution
2.00
1.75
1.50
1.25
1.00
0.75
0.50
0.25
0.00
−0.075
−0.050
−0.025
0.000
0.025
0.050
0.075




```
--- He Initialization (Validation) ---
Accuracy : 0.7547
F1 Score : 0.8103
Log Loss : 0.5149
-------------------------------------
```

```
Plotting Weights and Gradients distribution (He Initialization):
```



Inisialisasi	Accuracy	F1-score	Log Loss
Zero	0.6153	0.7619	0.6663
Random Uniform	0.7600	0.8115	0.5049
Random Normal	0.7540	0.8083	0.5110
Xavier	0.7573	0.8112	0.5094
He	0.7547	0.8103	0.5149

Analisis

- Zero initialization paling buruk
- Random uniform terbaik pada eksperimen ini
- Xavier dan he cukup dekat, tapi tidak melampaui random uniform

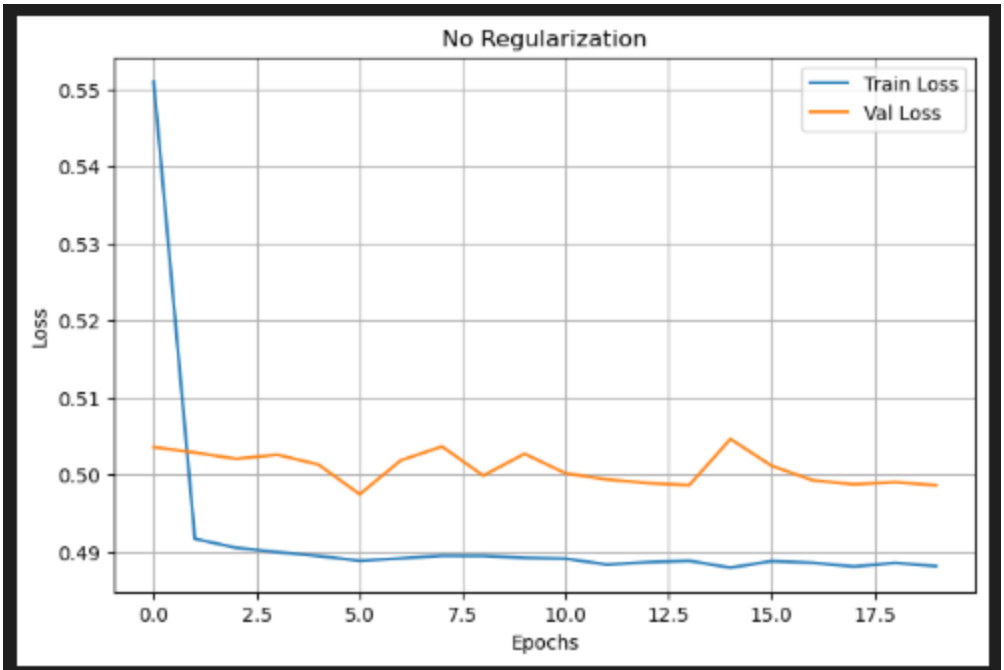
3.5 Pengaruh Regularisasi

Informasi

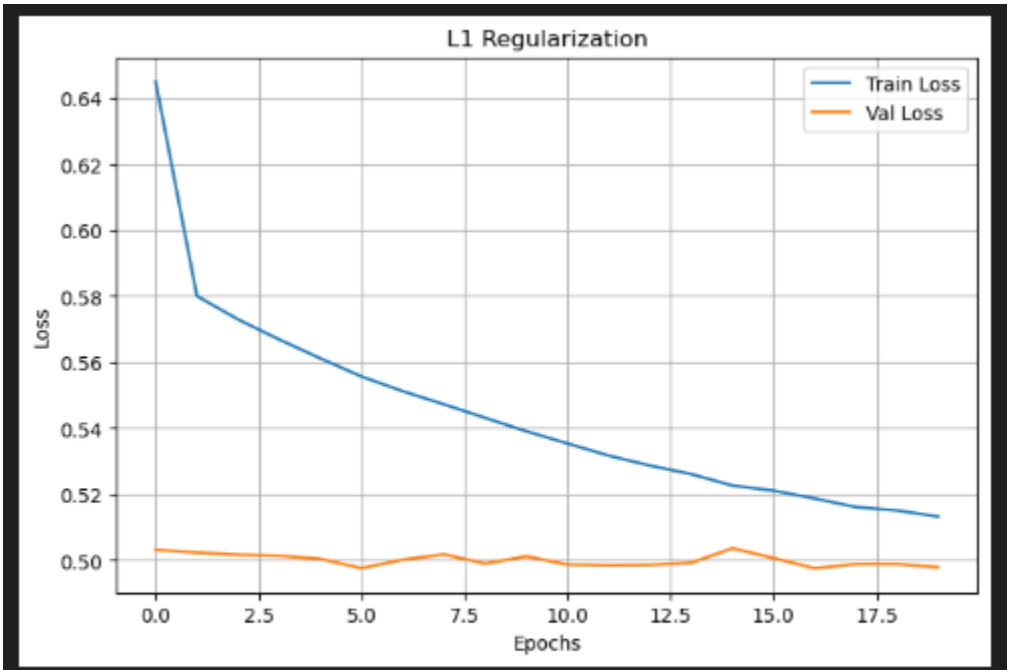
Dibandingkan:
- Tanpa regularisasi
- L1
- L2

Lambda = 0.001

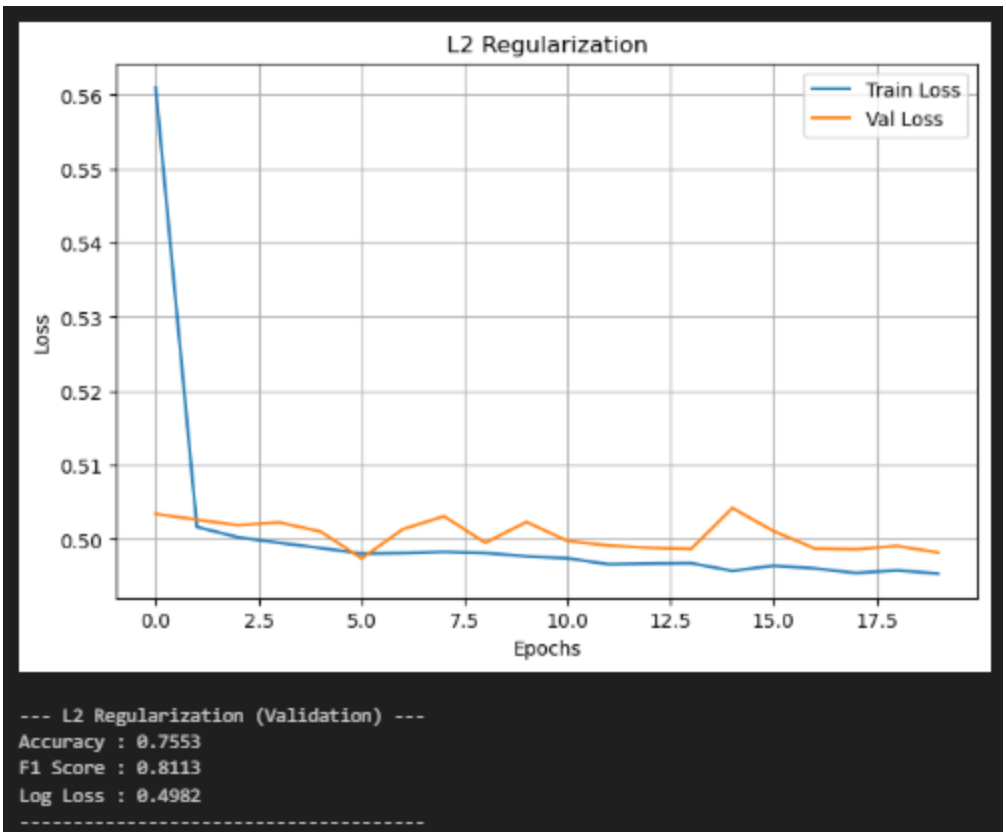
Grafik




```
--- No Regularization (Validation) ---
Accuracy : 0.7553
F1 Score : 0.8111
Log Loss : 0.4986
--------------------------------------
```




```
--- L1 Regularization (Validation) ---
Accuracy : 0.7600
F1 Score : 0.8158
Log Loss : 0.4979
--------------------------------------
```



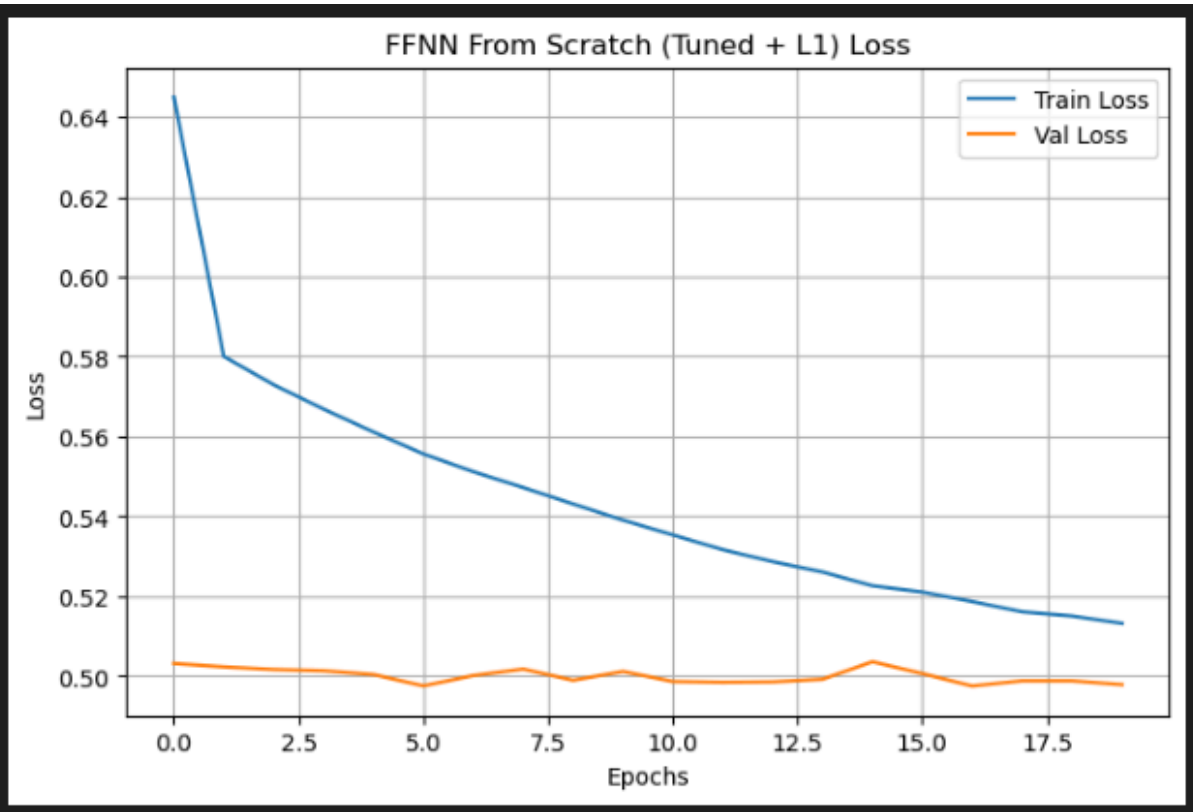
Regularisasi	Accuracy	F1-score	Log Loss
Tanpa Regularisasi	0.7553	0.8111	0.4986
L1	0.7600	0.8158	0.4979
L2	0.7553	0.8113	0.4982

Analisis

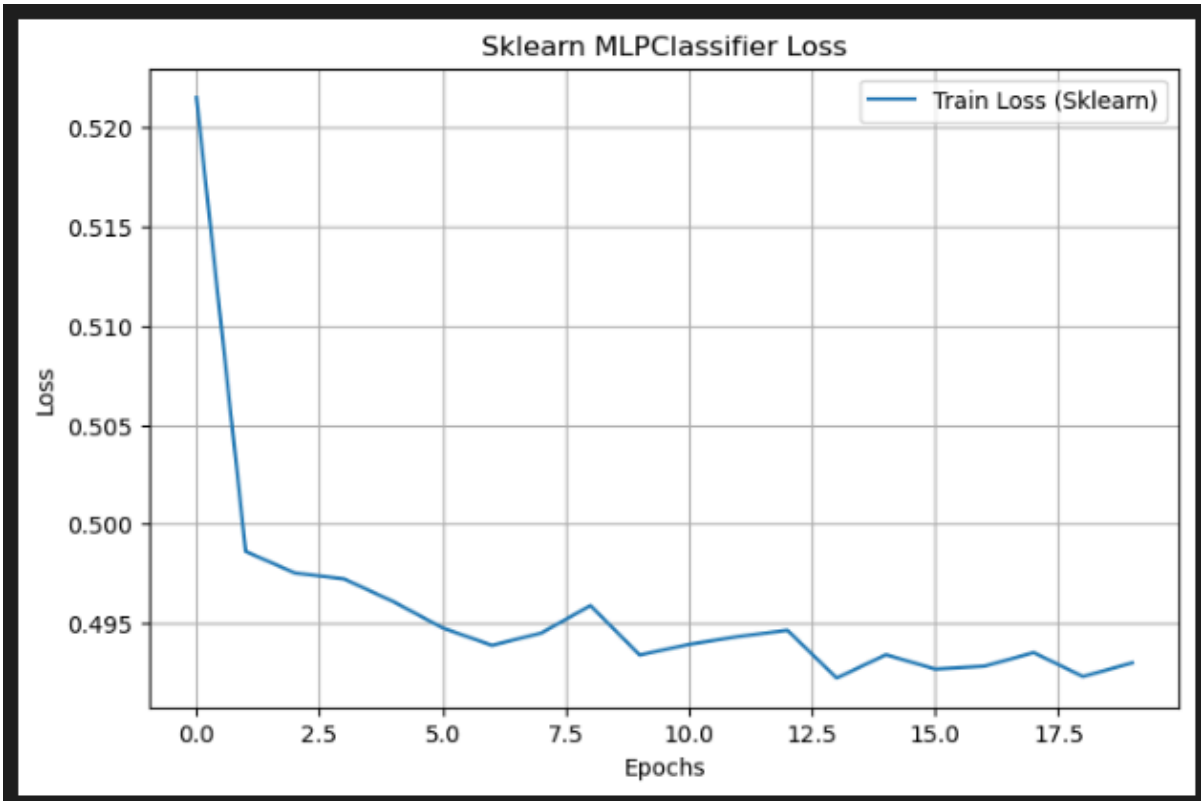
- L1 memberi hasil terbaik
- L2 mirip tanpa regularisasi
- Regularisasi membantu menjaga generalisasi, terutama L1 pada eksperimen ini

3.6 Perbandingan Model Terbaik dengan Sklearn MLPClassifier

Informasi
Model terbaik from scratch dibandingkan dengan *MLPClassifier* dari sklearn.
Grafik




```
--- FFNN From Scratch Tuned + L1 (Test) ---
Accuracy : 0.7567
F1 Score : 0.8163
Log Loss : 0.4884
-----------------------------------------
```




```
--- Sklearn MLPClassifier ---
Accuracy : 0.7620
F1 Score : 0.8110
Log Loss : 0.4925
----------------------------
```

Model	Accuracy	F1-score	Log Loss
FFNN from Scratch Terbaik	0.7567	0.8163	0.4884
Sklearn MLPClassifier	0.7620	0.8110	0.4925

Analisis

- Sklearn unggul tipis di accuracy
- Model from scratch unggul di F1 dan log loss
- Implementasi manual sudah cukup kompetitif dibandingkan library siap pakai

BAB IV
KESIMPULAN DAN SARAN

4.1 Kesimpulan

Berdasarkan hasil implementasi dan serangkaian percobaan yang telah dilakukan, model *Feedforward Neural Network* from scratch berhasil dibangun dan dapat digunakan untuk melakukan klasifikasi biner pada dataset *Global Student Placement & Salary Dataset*. Implementasi yang dibuat telah mencakup komponen utama neural network, yaitu *forward propagation*, *backward propagation*, pembaruan bobot dengan *gradient descent*, beberapa fungsi aktivasi, beberapa metode inisialisasi bobot, serta regularisasi L1 dan L2. Secara umum, model dapat dilatih dengan baik dan menunjukkan performa yang kompetitif.

Dari percobaan pengaruh *width* dan *depth*, diperoleh bahwa konfigurasi hidden layer dengan *width* 16 dan *depth* 2 memberikan hasil yang paling seimbang. Penambahan jumlah neuron atau jumlah layer yang terlalu besar tidak selalu meningkatkan performa, bahkan pada beberapa kasus justru menyebabkan penurunan kualitas model. Hal ini menunjukkan bahwa kompleksitas arsitektur perlu disesuaikan dengan karakteristik data agar model tetap efektif dan tidak mudah mengalami penurunan generalisasi.

Pada percobaan fungsi aktivasi, terlihat bahwa fungsi aktivasi memberikan pengaruh besar terhadap kestabilan proses pelatihan dan performa akhir model. Aktivasi linear, tanh, dan ELU memberikan hasil yang relatif lebih baik, sedangkan sigmoid menunjukkan performa yang paling lemah karena proses konvergensinya lebih lambat dan nilai loss akhirnya lebih tinggi. Pada percobaan *learning rate*, nilai 0.1 memberikan hasil terbaik dibandingkan 0.01 dan 0.001. *Learning rate* yang terlalu kecil membuat proses pembelajaran berjalan lambat sehingga model sulit mencapai performa optimal dalam jumlah epoch yang sama.

Pada percobaan inisialisasi bobot, metode *random uniform* memberikan performa terbaik, sedangkan *zero initialization* menghasilkan performa paling buruk. Hasil ini menunjukkan bahwa pemilihan inisialisasi bobot sangat penting karena memengaruhi kemampuan model untuk belajar sejak awal pelatihan. Sementara itu, pada percobaan regularisasi, metode L1 memberikan hasil terbaik dibandingkan tanpa regularisasi maupun L2. Hal ini menunjukkan bahwa regularisasi dapat membantu menjaga kemampuan generalisasi model, terutama ketika digunakan dengan konfigurasi yang sesuai.

Jika dibandingkan dengan *MLPClassifier* dari *scikit-learn*, model terbaik hasil implementasi from scratch menunjukkan performa yang cukup kompetitif. Meskipun *scikit-learn* sedikit lebih unggul pada accuracy, model from scratch masih mampu memberikan hasil yang baik pada metrik F1-score dan *log loss*. Dengan demikian, implementasi yang dibuat dapat dikatakan berhasil memenuhi tujuan tugas besar, baik dari sisi fungsionalitas maupun dari sisi kualitas hasil pelatihan model.

4.2 Saran

Untuk pengembangan lebih lanjut, percobaan dapat diperluas dengan menguji lebih banyak kombinasi hyperparameter, seperti jumlah epoch, batch size, variasi learning rate yang lebih rinci, serta nilai lambda regularisasi yang berbeda. Selain itu, analisis juga dapat ditingkatkan dengan mencoba dataset lain atau menggunakan validasi yang lebih kuat agar hasil evaluasi model menjadi lebih menyeluruh. Dari sisi implementasi, model juga masih dapat dikembangkan agar mendukung optimizer yang lebih baik seperti Adam atau RMSProp sehingga proses pelatihan dapat menjadi lebih stabil dan efisien.

Pembagian Tugas

Nama	Tugas
Amira Izani	- Laporan - Pengembangan `ffnn.py` - Forward-backward propagation - Update parameter dan regularisasi - Debugging feedforward
Benedictus Nelson	- Laporan - Pengembangan `experiment.ipynb` - Preprocessing dan evaluasi - Running skenario pengujian - Visualisasi hasil dan benchmarking sklearn - Debugging training

Lampiran

Repository: https://github.com/amiraIzani/LimaDua

Form Penggunaan AI:

FORMAT DEKLARASI PENGGUNAAN AI
SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, kami:

Nama/NIM : Amira Izani (13523143), Benedictus Nelson (13523150)
Fakultas/Sekolah : Sekolah Teknik Elektro dan Informatika - Komputasi
Kode Mata Kuliah : IF3270
Nama Mata Kuliah : Pembelajaran Mesin
Judul Tugas/Proposal : Pembuatan Feedforward Neural Network *from scratch*

Menyatakan bahwa kami menggunakan AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Tidak	-
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Ya	Membantu merancang struktur laporan
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Ya	Membantu pencarian referensi dan validasi desain serta rencana struktur awal pengerjaan
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Tidak	-

Lainnya, Jelaskan:		

Kami juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Jatinangor, 18 Maret 2026

Referensi

[1] Tim Pengajar IF3270, “Artificial Neural Network (ANN),” Institut Teknologi Bandung, Materi Perkuliahan IF3270 Pembelajaran Mesin, 2026. [Online]. Available: https://edunex.itb.ac.id/courses/88964/preview/423338 . [Accessed: Mar. 2026].

[2] GeeksforGeeks, “Xavier Initialization - GeeksforGeeks,” [Online]. Available: https://www.geeksforgeeks.org/xavier-initialization/ . [Accessed: Mar. 2026].

[3] GeeksforGeeks, “Activation Functions in Neural Networks,” [Online]. Available: https://www.geeksforgeeks.org/activation-functions-neural-networks/ . [Accessed: Mar. 2026].

[4] E. Bendersky, “The Softmax function and its derivative,” Eli Bendersky's website, [Online]. Available: https://eli.thegreenplace.net/2016/the-softmax-function-and-its-derivative/ . [Accessed: Mar. 2026].

[5] Tim Asisten IF3270, “Spesifikasi Tugas Besar 1: Feedforward Neural Network,” Program Studi Teknik Informatika, Institut Teknologi Bandung, [Online]. Available: https://docs.google.com/document/d/13uWIVpfMsQbOC1bOOgGjBdlQEhgp92RJ/edit . [Accessed: Mar. 2026]